

Aero 626 Project Final Report

State Estimation for Lunar Transfer Trajectories

Aadarsh Sudanagunta

December 2025

Abstract

This report discusses the state estimation of a spacecraft in a Trans-Lunar trajectory subject to nonlinear dynamics and measurements. Batch Least Squares (BLS) and Unscented Kalman Filtering (UKF) will be used due to historical relevance and derivative free methods, respectively. These two methods were analyzed for their robustness with poor initial state estimates and measurement outages as well as computational run time. Overall, the BLS performed worse than the UKF, but a modified form was able to match the UKF. The modified BLS has lower estimation error, but is slower, resulting in the UKF being chosen for real-time applications.

1 Introduction

For the first time since the Apollo missions, humanity is making its way back to the Moon. NASA's Artemis program aims to establish a sustained human presence on the lunar surface with commercial entities planning cargo delivery, resource extraction, and even tourism missions. Supporting these challenging missions requires the Lunar Gateway, a space station in orbit around the Moon that will act as the in-orbit base for vehicle docking, surface operations, and space technology testing. All of these missions have a common problem: spacecraft trajectories. Spacecraft have to navigate their way from Earth to the Moon and Gateway, all while traversing a region with a highly nonlinear gravitational field, resulting in accompanying complex and nonlinear trajectories.

During the coast phase, a multi-day journey between Trans-Lunar Injection (TLI) and Lunar Orbit Insertion (LOI), spacecraft have to navigate between two gravitational regimes, Earth and Moon. Unlike simple two-body orbital mechanics, where spacecraft orbit a central body in relatively predictable elliptical paths, three-body dynamics introduce fundamental nonlinearities. As the coast phase progresses, the gravitational influence of Earth gradually weakens, while the Moon's influence strengthens, and the spacecraft must pass through regions of instability, where small deviations can result in drastically different trajectories. Navigation during this phase can be particularly challenging because the spacecraft starts to get too far from Earth for high-frequency tracking while simultaneously not being close enough to the Moon to be supported by lunar infrastructure. Ground stations on Earth can provide range and Doppler (range rate), but measurements can be sparse due to station availability and communication constraints.

Accurate real-time state estimation is critical during this phase. In unexpected mission scenarios, such as thruster misfire or sensor degradation, without accurate knowledge of the spacecraft's current state, determining the required corrective maneuvers can become impossible. In scenarios where there is a large enough deviation from the intended trajectory, the spacecraft may completely miss the target destination, which is especially disastrous for crewed missions, such as those in the Artemis program.

There are several methods that can be used to estimate the dynamical systems. One of the most popular and oldest methods, is Least Squares (LS), or in a form better suited for lots of data, Batch Least Squares (BLS). Historically, least squares was used for orbit determination (OD). However, least squares can have several drawbacks, such as high computational load and needing to iterate for nonlinear systems. As a result, least-squares can be better for updating with large batches of data, i.e. data processing for once per revolution datasets. Due to this computational load limitation, least squares may not be the best method for real-time applications, such as OD during unexpected perturbations or nonlinear behavior [1]. Accordingly, Kalman filters have been the foundation of modern OD, especially in real-time for spacecraft navigation systems. There are several different types of Kalman filtering: classic, extended (EKF), and unscented (UKF).

The classic Kalman filter has a similar problem as BLS, in that it was made for linear problems, and so it doesn't work for nonlinear problems, such as OD. As a result, EKF and UKF have been more of a focus as alternatives to BLS. NASA conducted a study in 1991 that compared satellite orbit determination using BLS and EKF [2]. The study noted that results had to be split, as the EKF had not reached steady state during the early phases. As a result, the EKF performed poorly for the initial part, but was significantly better once it had reached steady state. However, EKF is still not perfect, as it is still not a solution for highly nonlinear systems, particularly in three-body dynamics, where the system behavior changes character as the spacecraft transitions between gravitational spheres of influence. This problem with EKF is what makes UKF a potentially perfect fit for OD. Unlike EKF, UKF doesn't rely on any linear modeling of the dynamics or measurements, allowing it to be highly flexible in nonlinear problems. A January 2005 study published to *Acta Astronomica Sinica* showed that the performance of UKF was superior to that of EKF for GPS based satellite OD [3]. Additionally, UKF typically achieves its higher accuracy while maintaining similar computational costs to EKF. As a result, UKF shows strong promise to be the best method for OD.

2 Problem Statement

This project proposes to estimate the position and velocity of a spacecraft during a lunar transfer trajectory using range and Doppler measurements from ground-based tracking stations. The estimation process is inherently nonlinear due to three factors: the three-body gravitational dynamics from the Circular Restricted Three-Body Problem (CR3BP), the range measurements, and the range rate measurements. Two estimation approaches will be implemented and compared, the historically used BLS, and the linearization free Kalman Filter, the UKF. The project will examine how these different methods handle the nonlinearities present during the transition between the Earth and Moon gravitational regimes as well as the robustness under potential problems, such as measurement outages.

2.1 System Dynamics

Simulating the motion of bodies in space often requires restricting scope and fidelity. When analyzing the motion of spacecraft and satellites orbiting Earth, several other bodies, such as the Sun, Moon, or even Jupiter could be taken into account to more accurately model the motion. However, these bodies may not have significant impact on the overall motion, so removing them from the scope of the analysis can have results that are only marginally different. In the case of missions to the Moon, it is imperative to keep the Moon in the scope of the simulation, as spacecraft will be traversing through the Cislunar space. Since the Moon adds a level of complexity, other restrictions can be made to help simplify the problem. A common approach to analysis of motion in the Cislunar regime is the Circular Restricted 3 Body Problem (CR3BP). In this methodology, the main coordinate frame is called the CR3BP frame, or the synodic frame, as it is centered at the center of mass (barycenter) of the two primary bodies, and rotates with the bodies as they move around the barycenter [4].

To model the spacecraft's motion for this project, a nondimensional form of the Earth-Moon CR3BP approach will be taken. In the nondimensional form, dimensional values (x_D) are converted to nondimensional values (x_{ND}) using the following scale factors:

$$\text{distance } d_D = LU d_{ND} \quad (1)$$

$$LU = r_{Earth-Moon} = 384400000 \text{ m} \quad (2)$$

$$\text{time } t_d = TU t_{ND} \quad (3)$$

$$TU = \sqrt{\frac{LU^3}{G(M_E + M_M)}} \quad (4)$$

$$\text{velocity } v_d = VU v_{ND} \quad (5)$$

$$VU = LU/TU \quad (6)$$

In addition to the scale factors, the system will be parametrized using the ratio of the mass, μ :

$$\mu = \frac{m_2}{m_1 + m_2} \quad \text{where } m_1 = M_E, m_2 = M_M \quad (7)$$

$$\mu_1 = 1 - \mu \quad (8)$$

$$\mu_2 = \mu \quad (9)$$

Once the system has been nondimensionalized, the next step is to model the dynamics of the vehicle and find the equations of motion using the Lagrangian and Euler-Lagrange equations. Typically, the Lagrangian is defined in the inertial frame as the kinetic energy minus the potential energy, $\mathcal{L} = T - U$. Converting this equation to the synodic frame and writing the Euler-Lagrange equations yields the following equations of motion.

$$\ddot{x} - 2\dot{y} = -\frac{\partial \bar{U}}{\partial x} \quad (10)$$

$$\ddot{y} + 2\dot{x} = -\frac{\partial \bar{U}}{\partial y} \quad (11)$$

$$\ddot{z} = -\frac{\partial \bar{U}}{\partial z} \quad (12)$$

where the effective potential $\bar{U}$ is:

$$\bar{U}(x, y, z) = -\frac{1}{2}(x^2 + y^2) - \frac{\mu_1}{r_1} - \frac{\mu_2}{r_2} - \frac{1}{2}\mu_1\mu_2 \quad (13)$$

with:

$$r_1 = \sqrt{(x + \mu_2)^2 + y^2 + z^2} \quad (14)$$

$$r_2 = \sqrt{(x - \mu_1)^2 + y^2 + z^2} \quad (15)$$

In the synodic frame, the state of the spacecraft at any time is defined using its position and velocity, or combined as a single vector:

$$\mathbf{x}(t) = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T \in \mathbb{R}^6 \quad (16)$$

Utilizing an initial state, the state of the spacecraft can be simulated as it travels through the cislunar space by the dynamics model:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) \quad (17)$$

Where $\mathbf{f}(\mathbf{x}(t))$ is created through the application of the equations of motion.

2.2 Measurement Model

To accurately estimate the state of the spacecraft in real time, measurements are taken from ground stations on Earth. However, these ground stations do not explicitly measure the state of the spacecraft, instead, they give the range (ρ) and Doppler ($\dot{\rho}$) relative to the ground station, which has position $\mathbf{r}_s$ and velocity $\dot{\mathbf{r}}_s$ in the synodic frame. Since the ground stations are fixed to the Earth as it rotates, their positions and velocities are constantly changing. Therefore, it is necessary to account for them in the measurements. The measurements can be modeled according to:

$$\rho_k = \|\mathbf{r}_k - \mathbf{r}_s(t_k)\| + v_{\rho,k} \quad (18)$$

$$\dot{\rho}_k = \frac{(\mathbf{r}_k - \mathbf{r}_s(t_k))^T (\dot{\mathbf{r}}_k - \dot{\mathbf{r}}_s(t_k))}{\|\mathbf{r}_k - \mathbf{r}_s(t_k)\|} + v_{\dot{\rho},k} \quad (19)$$

where $\mathbf{r}_k$ is the spacecraft position and $v_{\rho,k}$ and $v_{\dot{\rho},k}$ are measurement noises with covariance. In a simpler form, the measurement model can be written as:

$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, t_k) + \mathbf{v}_k \quad (20)$$

3 Methods

3.1 Range and Doppler Standard Deviations

In this project, a key component is the use of ground stations, the Deep Space Network (DSN), to get measurements of range and Doppler. However, a key part of the estimation methods used in this project is modeling the covariance/standard deviation throughout the estimation process. As a result, it is important to find what the typical standard deviations in measurements by the DSN are.

There are two main methods that are used to determine range and Doppler: one-way ranging and two-way ranging. One-way ranging measures range and Doppler through sending a signal from an in-orbit vehicle to a ground station. Two-way ranging, on the other hand, uses round trip information, making it more accurate and useful. These two methods have different advantages, but two-way ranging will be used for this project, as it is the most common method used today. Additionally, each of these methods has different calculations that need to be done to determine the standard deviations, as two-way is susceptible to downlink and uplink noise contributions.

For range, it is possible to calculate the standard deviation through the process described in Ref. [5]:

$$\sigma_\rho = \frac{c\sigma_\tau}{2} \quad (21)$$

In equation (21), c is the speed of electromagnetic waves in vacuum, and σ_τ is the standard deviation of the time delay. This value can be found according to equation (22) using the frequency of the range clock (f_R), the ratio of the downlink ranging signal power to the one-sided noise spectral density of the downlink receiver ($\frac{P_R}{N_0}|_D$), the integration time (T_1).

$$\sigma_\tau = \frac{1}{2\pi f_R \sqrt{2T_1 \frac{P_R}{N_0}|_D}} \quad (22)$$

Using equation (22) for a "1-MHz range clock, a standard deviation σ_τ of 6 ns is obtained" when $T_1 = 175$ s and $\frac{P_R}{N_0}|_D = 3$ dB-Hz [5]. Plugging in 6 ns into equation (21) yields a $\sigma_\rho \approx 1$ m.

Doppler standard deviation calculation is more complicated, and is dependent on several factors, such as the solar phase scintillation. Overall, the error variance σ_V^2 , can be modeled according to (23), as the sum of the contributions of white (thermal) noise (σ_{VN}), phase noise of frequency sources (σ_{VF}), and phase scintillation (σ_{VS}).

$$\sigma_V^2 = \sigma_{VN}^2 + \sigma_{VF}^2 + \sigma_{VS}^2 \quad (23)$$

For simplicity and scope reduction, only the white noise will be taken into account. In two-way ranging, the white noise is composed of the uplink (σ_{VNU}^2) and downlink (σ_{VND}^2) components according to equations (24) and (25).

$$\sigma_{VNU}^2 \approx \frac{1}{2} \left(\frac{c}{2\pi f_C T} \right)^2 \frac{G^2}{\rho_{TR}} \quad (24)$$

$$\sigma_{VND}^2 = \frac{1}{2} \left(\frac{c}{2\pi f_C T} \right)^2 \frac{1}{\rho_L} \quad (25)$$

In these equations, c is the speed of light, f_C is the downlink carrier frequency, T is the integration time, G is the transponding ratio, ρ_{TR} is the signal-to-noise ratio in the transponder's carrier loop, and ρ_L is the signal-to-noise ratio in the downlink carrier loop. After some estimation, common values of $\sigma_V N$ are less than 0.1 mm/s [6].

3.2 Batch Least Squares

BLS is a classical approach to OD, and processes all measurements simultaneously to minimize the weighted sum of squared residuals. Given a set of measurements $\mathbf{z}$ over a time span and an initial reference state $\mathbf{x}_0$, the estimate $\hat{\mathbf{x}}_0$ of the initial state is found by minimizing the cost function. This nonlinear least squares problem will be solved iteratively using the Gauss-Newton method. The batch approach provides optimal estimates when all of the measurements are available and the dynamics model is accurate, but can struggle providing real time estimates if convergence time is high. This will serve as the reference solution for comparing the other methods. The implementation of the BLS will follow the iterative method discussed in this class, but will include one modification, line search, to help converge and reduce compute time.

In the basic BLS using the Linear Unbiased Minimum Variance Estimate (LUMVE), the process starts with an initial reference state $\mathbf{x}_0^*$ and covariance $\bar{\mathbf{P}}_{xx,0}$ at time t_0 . Then, the solver enters the accumulation phase, where it gathers information from all the data points using the state transition matrix Φ_l , the measurement noise covariance $\mathbf{P}_{vv,l}$, the measurement function $\mathbf{h}$, and the measurement Jacobian $\tilde{\mathbf{H}}$.

$$\lambda_l = \lambda_{l-1} + \left[\tilde{\mathbf{H}}(\mathbf{x}_l^*) \Phi(t_l, t_0) \right]^T \mathbf{P}_{vv,l}^{-1} [\mathbf{z}_l - \mathbf{h}(\mathbf{x}_l^*)] \quad \text{at } t_l \quad (26)$$

$$\mathbf{\Lambda}_l = \mathbf{\Lambda}_{l-1} + \left[\tilde{\mathbf{H}}(\mathbf{x}_l^*) \Phi(t_l, t_0) \right]^T \mathbf{P}_{vv,l}^{-1} \left[\tilde{\mathbf{H}}(\mathbf{x}_l^*) \Phi(t_l, t_0) \right] \quad \text{at } t_l \quad (27)$$

Equations (26) and (27) are then used to solve for the estimate and covariance for that iteration, k .

$$\mathbf{\Lambda}_k \delta \hat{\mathbf{x}}_{0,k} = \boldsymbol{\lambda}_k \quad (28)$$

$$\mathbf{P}_{xx,0,k} = \mathbf{\Lambda}_k^{-1} \quad (29)$$

$$\mathbf{x}_{0,k+1}^* = \mathbf{x}_{0,k}^* + \delta \hat{\mathbf{x}}_{0,k} \quad (30)$$

To iterate till convergence, the BLS continues with updated reference state and covariance until the step size $\delta \hat{\mathbf{x}}_0$ is less than a prespecified tolerance and stops monotonically decreasing in magnitude.

3.2.1 Line Search

In some nonlinear problems, BLS can have a hard time converging. To aid the BLS in achieving convergence, the typical Gauss-Newton iteration is modified to include an additional line search parameter, α .

$$\mathbf{x}_{0,k+1}^* = \mathbf{x}_{0,k}^* + \alpha(\delta \hat{\mathbf{x}}_{0,k}) \quad \alpha \in (0, 1] \quad (31)$$

As α decreases, the step size between iterations decreases, allowing the solver to approach the solution without diverging and estimating an infeasible states. Typically, Armijo's condition is used to determine the value of α [7]. However, for simplicity in its current state, the BLS solver will use a two part definition for α . First, if $\|d\mathbf{x}_{0,k}\| > 0.5 * \|d\mathbf{x}_{0,1}\|$, then $\alpha = 0.5$. Secondly, if the step size starts diverging, i.e. starts growing compared to the previous step size, then $\alpha = 0.1$ to clamp the step size down hard and push the iteration back towards convergence. Together, these two values of α can significantly help the BLS reach its final estimate for the initial state.

3.3 Unscented Kalman Filter (UKF)

The UKF is on the newer side of Kalman filtering, coming after the classical KF and the EKF. The biggest strength of the UKF is that it does not rely on any creating Jacobians for the dynamics and measurement models. Instead, the UKF utilizes the unscented transform and a set of sigma points to approximate the target mean and covariance. For a system with n states, the $2n + 1$ sigma points are created as follows using the mean, $\mathbf{m}_x$, and the covariance, $\mathbf{P}_{xx}$, at step $k - 1$.

$$\boldsymbol{\chi}_{k-1}^{(0)} = \mathbf{m}_{x,k-1} \quad (32)$$

$$\chi_{k-1}^{(i)} = \mathbf{m}_{x,k-1} + \sqrt{n + \lambda} [\mathbf{S}_{xx,k-1}]_i \quad (33)$$

$$\chi_{k-1}^{(i)} = \mathbf{m}_{x,k-1} - \sqrt{n + \lambda} [\mathbf{S}_{xx,k-1}]_i \quad (34)$$

$[\mathbf{S}_{xx}]_i$ is i^{th} the column of the square root of the covariance created using Cholesky decomposition. λ is a parameter of the UKF which is typically defined as:

$$n + \lambda = \alpha^2(n + \kappa) \quad (35)$$

Conventionally, κ is set so that $n + \kappa = 3$; however, in systems with a large number of states, this may cause issues. Consequently, Ref. [8] and Ref. [9] were used to set $\alpha = 1$ and $\kappa = 0$. Utilizing these values, appropriate weights can be created for the mean, $w^{(m)}$, and covariance, $w^{(c)}$, with an additional parameter $\beta = 2$ for Gaussian distributions.

$$w_0^{(m)} = \frac{\lambda}{n + \lambda}, \quad w_i^{(m)} = \frac{\lambda}{2(n + \lambda)} \quad (36)$$

$$w_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta), \quad w_i^{(c)} = \frac{\lambda}{2(n + \lambda)} \quad (37)$$

Once the sigma points have been created, they go through equations (38) - (40) to propagate forward from the posterior at step $k - 1$ to the prior at step k .

$$\chi_k^{(i)} = \mathbf{f}(\chi_{k-1}^{(i)}) \quad (38)$$

$$\mathbf{m}_{x,k}^- = \sum_{i=0}^{2n} w_i^{(m)} \chi_k^{(i)} \quad (39)$$

$$\mathbf{P}_{xx,k}^- = \sum_{i=0}^{2n} w_i^{(c)} \left(\chi_k^{(i)} - \mathbf{m}_{x,k}^- \right) \left(\chi_k^{(i)} - \mathbf{m}_{x,k}^- \right)^T + \mathbf{P}_{ww,k-1} \quad (40)$$

Once the propagated mean and covariance have been calculated, they are used to compute new sigma points and weights that then go through the update phase to find the mean and covariance at the posterior of step k .

$$\mathbf{z}_k^{(i)} = \mathbf{h}(\chi_k^{(i)}) \quad (41)$$

$$\mathbf{m}_{z,k}^- = \sum_{i=0}^{2n} w_i^{(m)} \mathbf{z}_k^{(i)} \quad (42)$$

$$\mathbf{P}_{zz,k}^- = \sum_{i=0}^{2n} w_i^{(c)} \left(\mathbf{z}_k^{(i)} - \mathbf{m}_{z,k}^- \right) \left(\mathbf{z}_k^{(i)} - \mathbf{m}_{z,k}^- \right)^T + \mathbf{P}_{vv,k} \quad (43)$$

$$\mathbf{P}_{xz,k}^- = \sum_{i=0}^{2n} w_i^{(c)} \left(\boldsymbol{\chi}_k^{(i)} - \mathbf{m}_{x,k}^- \right) \left(\mathbf{z}_k^{(i)} - \mathbf{m}_{z,k}^- \right)^T \quad (44)$$

$$\mathbf{m}_{x,k}^+ = \mathbf{m}_{x,k}^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{m}_{z,k}^-) \quad (45)$$

$$\mathbf{P}_{xx,k}^+ = \mathbf{P}_{xx,k}^- - \mathbf{P}_{xz,k}^- \mathbf{K}_k^T - \mathbf{K}_k (\mathbf{P}_{xz,k}^-)^T + \mathbf{K}_k \mathbf{P}_{zz,k}^- \mathbf{K}_k^T \quad (46)$$

$$\mathbf{K}_k = \mathbf{P}_{xz,k}^- (\mathbf{P}_{zz,k}^-)^{-1} \quad (47)$$

This process is repeated for all measurements/steps available, and yields both the estimated mean and covariance as the filter goes through the measurements. Unlike BLS, the UKF does not need to iterate, and only runs through the available measurements once.

3.4 Measurement Outages

To evaluate the two methods' robustness, both will be tested using simulated measurement outages. To accomplish this, the BLS will not go through the accumulation process for λ and Λ for the missing measurements. The UKF has two steps, propagate and update, where the propagate phase does not need the measurements. As a result, the UKF will only go through the first phase, resulting in the posterior being set equal to the prior for the outage.

4 Analysis and Discussion

The BLS and UKF methods both resulted in successful filters. However, these methods need to be compared to find which is ultimately better. The BLS method will be split into with (BLSLS) and without (BLS) line search categories to understand its benefits, while the UKF will remain as one category. These filters will be evaluated based on two criteria: resilience with poor initial state estimates and measurement outages. To test the first criteria, each of the methods will be run with deviations in the initial state estimate from the truth of [10, 50, 1000] m in each position component and [1, 5, 100] m/s in each velocity component. There are three categories for this criteria, low, medium, and high, with each corresponding to a level in the available deviations. For the second category, the filters will be run at the highest deviation level they can work with, and data will be removed until the filters 'break'.

4.1 Trajectory and Measurements

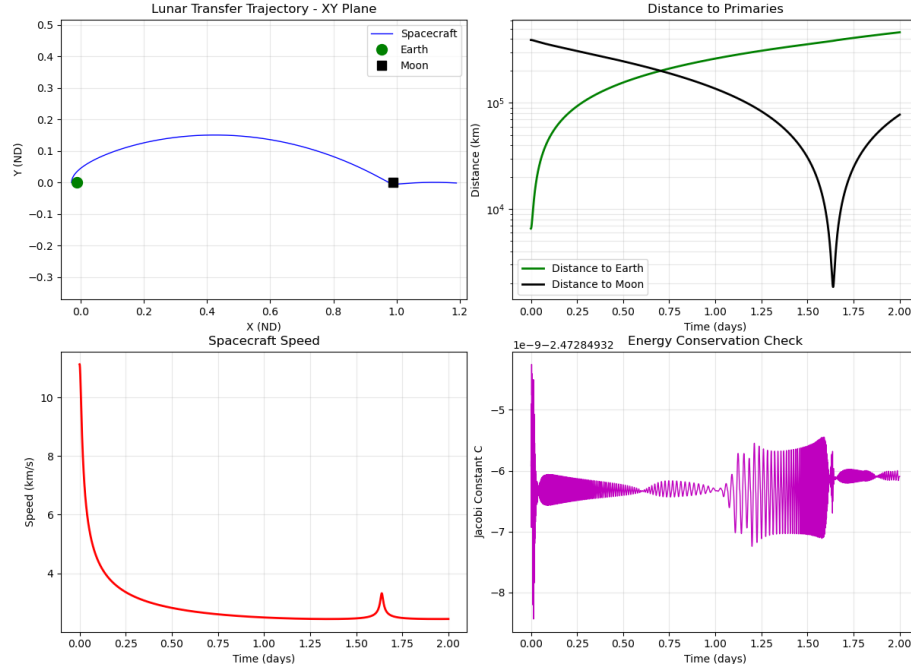


Figure 1: Simulated Trajectory and Verification

To evaluate the filters, an appropriate trajectory needs to be created. For this simulation, a spacecraft starts in LEO behind the Earth before doing an impulsive burn to put it into a Lunar Flyby trajectory. To verify that the trajectory was correctly created, the variation in the Jacobi constant was evaluated across the simulation time. This revealed minute changes on the order of 10^{-9} , showing that the Jacobi constant/Energy was held constant, resulting in a correctly generated trajectory. The biggest nonlinearities are extractable from Fig. 1 between 1.25 days and 1.65 days, as the Lunar gravitational regime starts taking over. Additionally, this period has the highly nonlinear motion of the flyby, which will be very useful for evaluating the filters' robustness and capability of handling highly nonlinear problems.

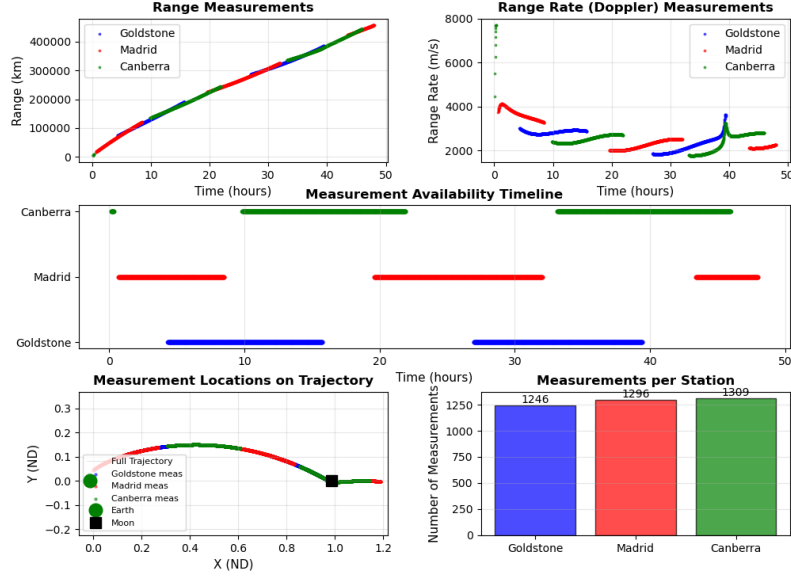


Figure 2: Trajectory Measurement Analysis

Utilizing the trajectory, appropriate measurements need to be created. The trajectory simulation utilizes the position/velocity state vector at each time, but these need to be converted to measurements. The measurement simulation takes all of the state values and runs them through the measurement model to create range and Doppler estimates from ground stations while adding in noise [10]. The Deep Space Network is designed for high coverage, and this is very useful for this simulation, as the three ground stations (Goldstone, Madrid, Canberra) are able to track the spacecraft across the entire trajectory besides a small portion in the beginning when the Earth is blocking all of the antenna [11].

4.2 Deviation Test

4.2.1 BLS without Line Search

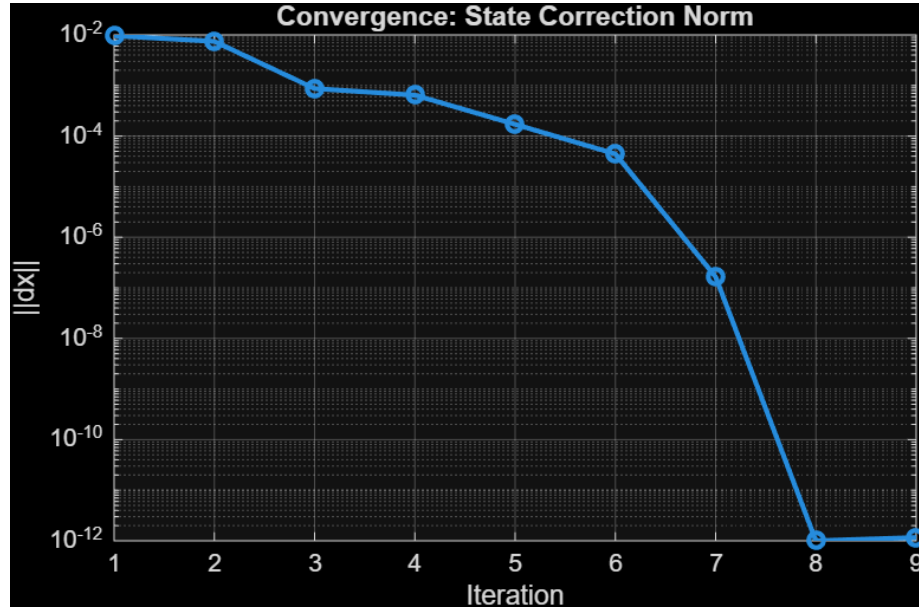


Figure 3: BLS Convergence with Low Deviation in Initial State

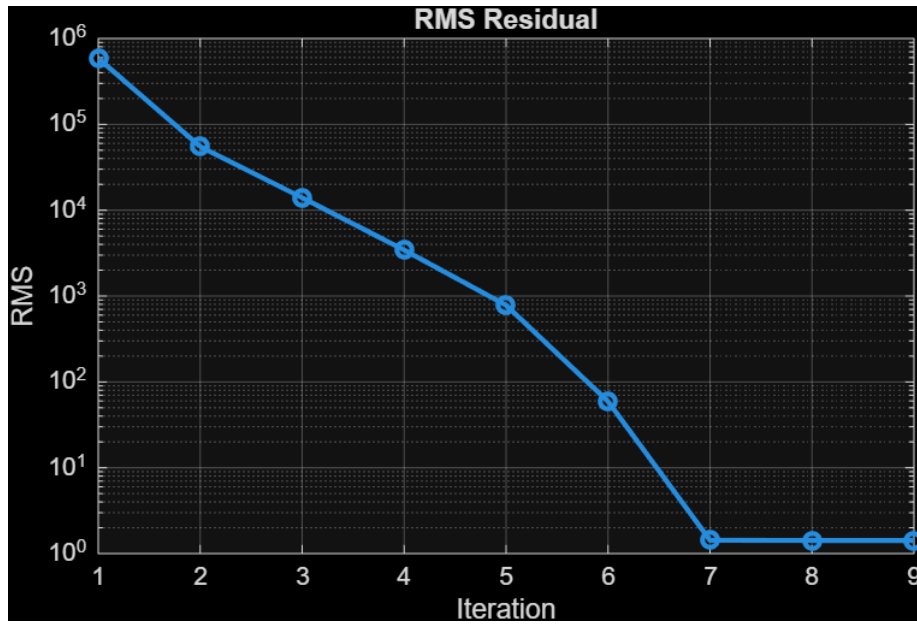


Figure 4: BLS RMS Residual with Low Deviation in Initial State

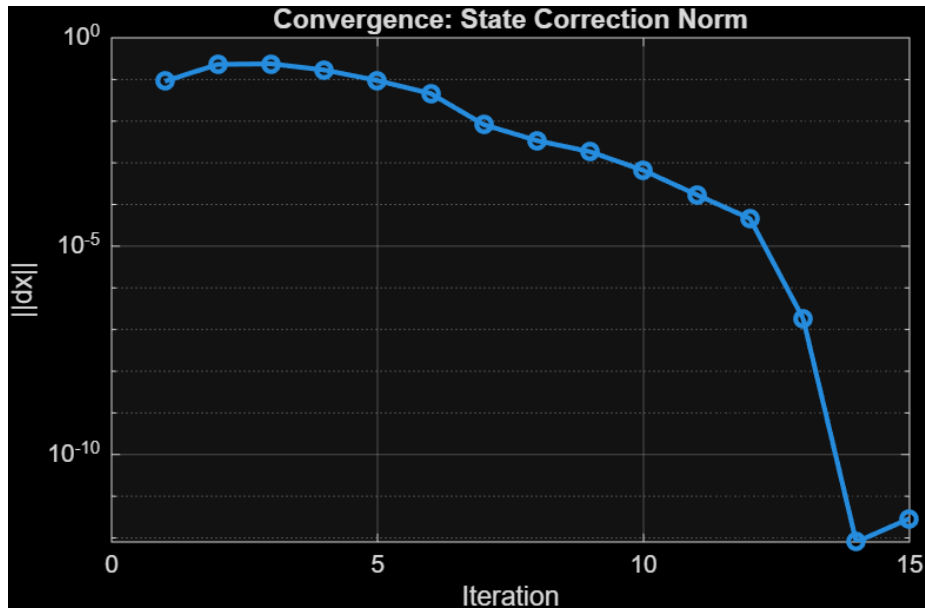


Figure 5: BLS Convergence with Medium Deviation in Initial State

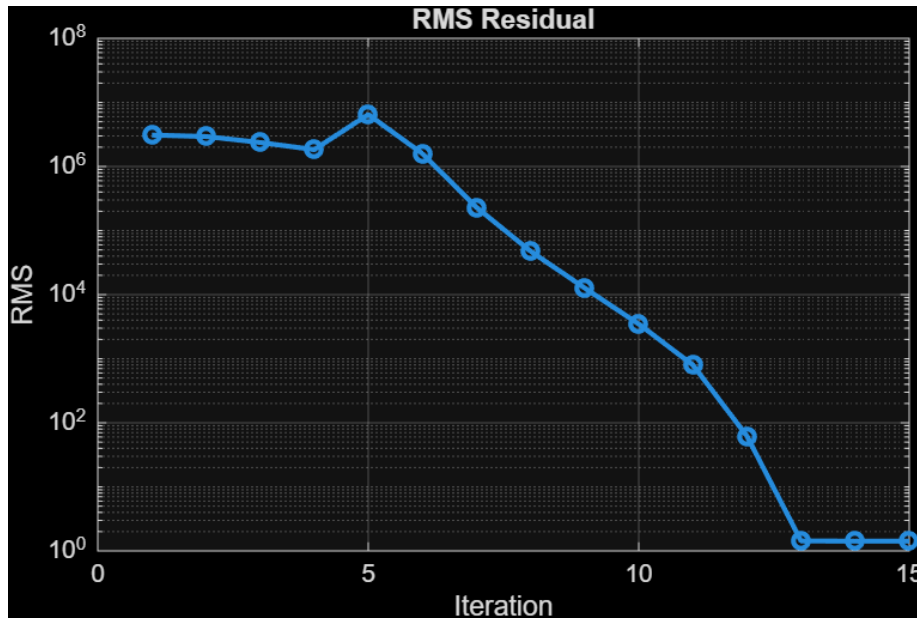


Figure 6: BLS RMS Residual with Medium Deviation in Initial State

Looking at the BLS without line search, it is seen how it visibly struggles. At the first level, the filter is able to converge quickly and easily. However, at the second level, it struggles, diverging a little initially before clamping back in and reaching convergence. The filter does not make it to the high level, and even pushing the deviations to 60 m and 6 m/s results in the filter failing and fully diverging.

4.2.2 BLS with Line Search

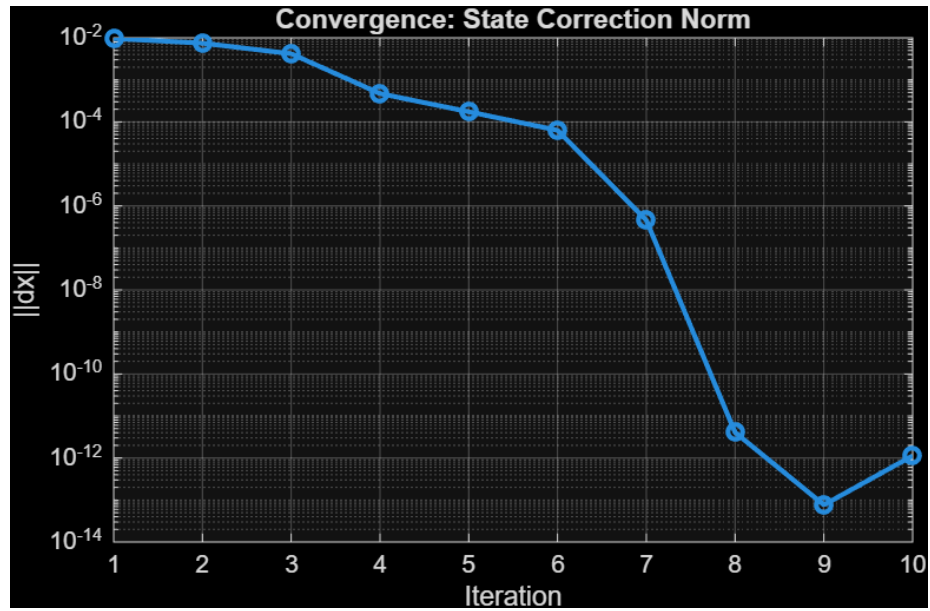


Figure 7: BLSLS Convergence with Low Deviation in Initial State

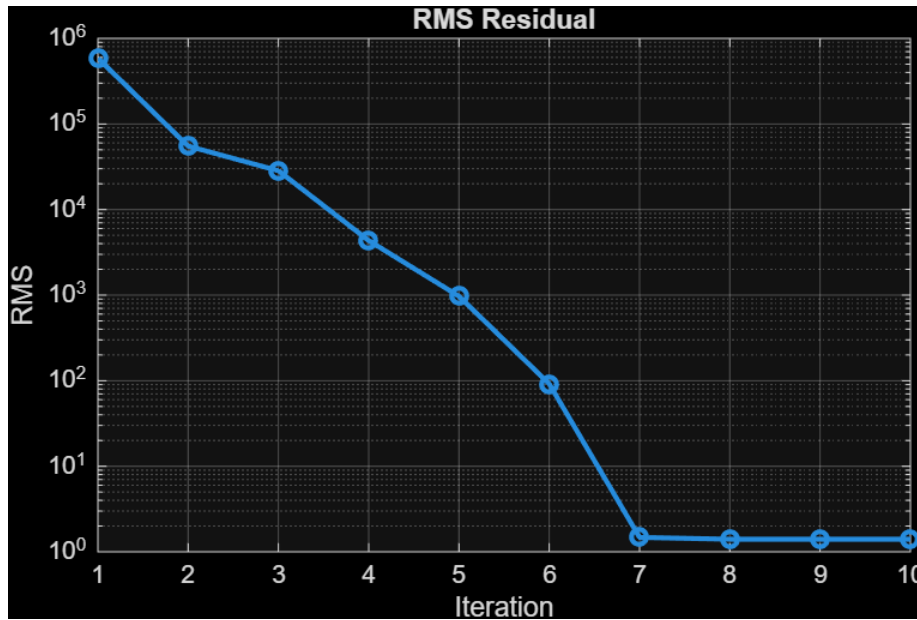


Figure 8: BLSLS RMS Residual with Low Deviation in Initial State

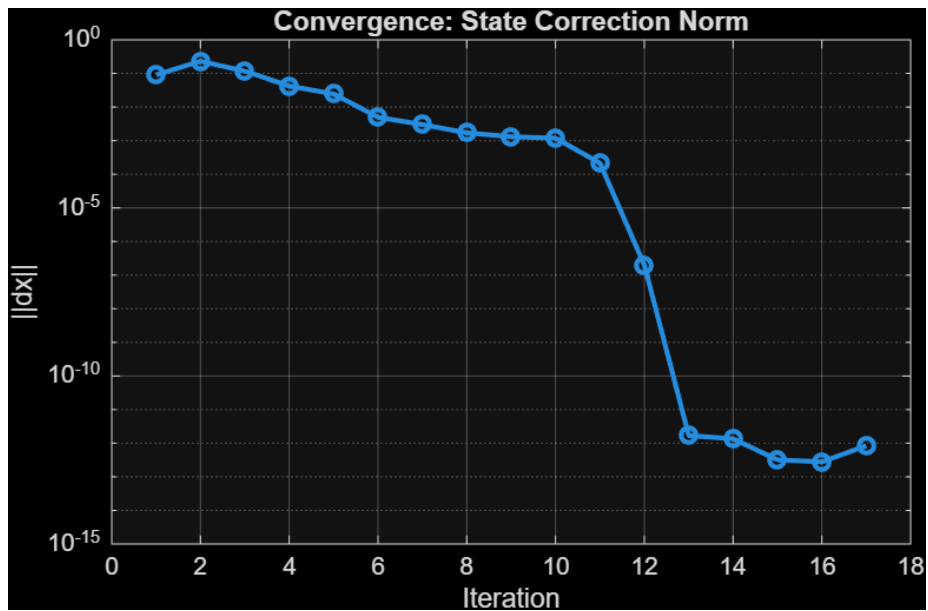


Figure 9: BLSLS Convergence with Medium Deviation in Initial State

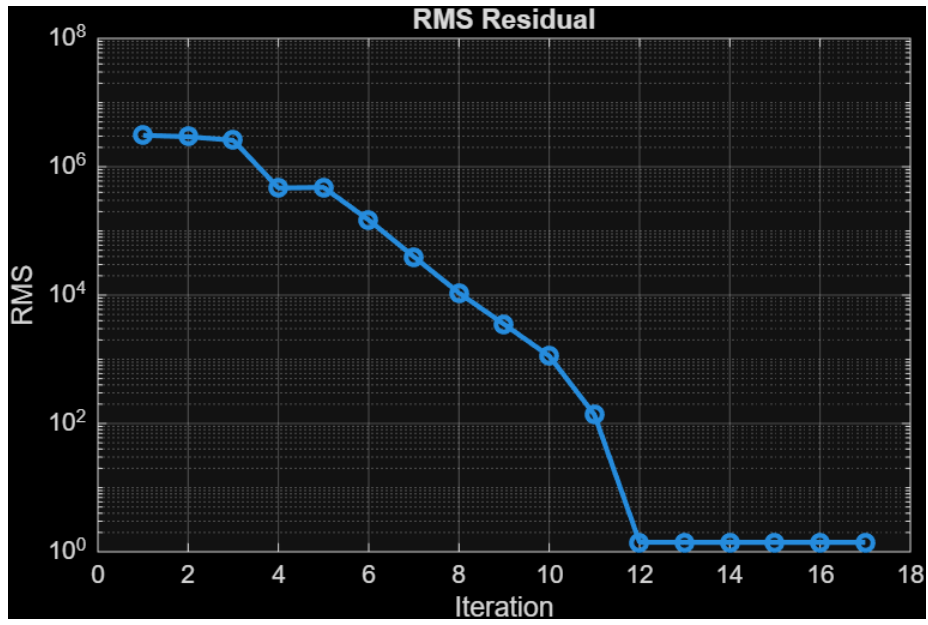


Figure 10: BLSLS RMS Residual with Medium Deviation in Initial State

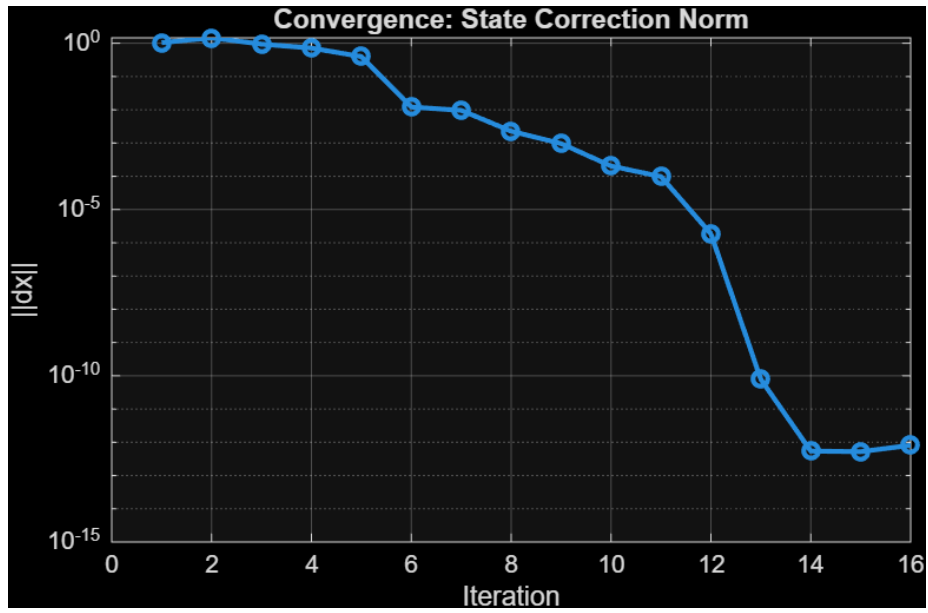


Figure 11: BLSLS Convergence with High Deviation in Initial State

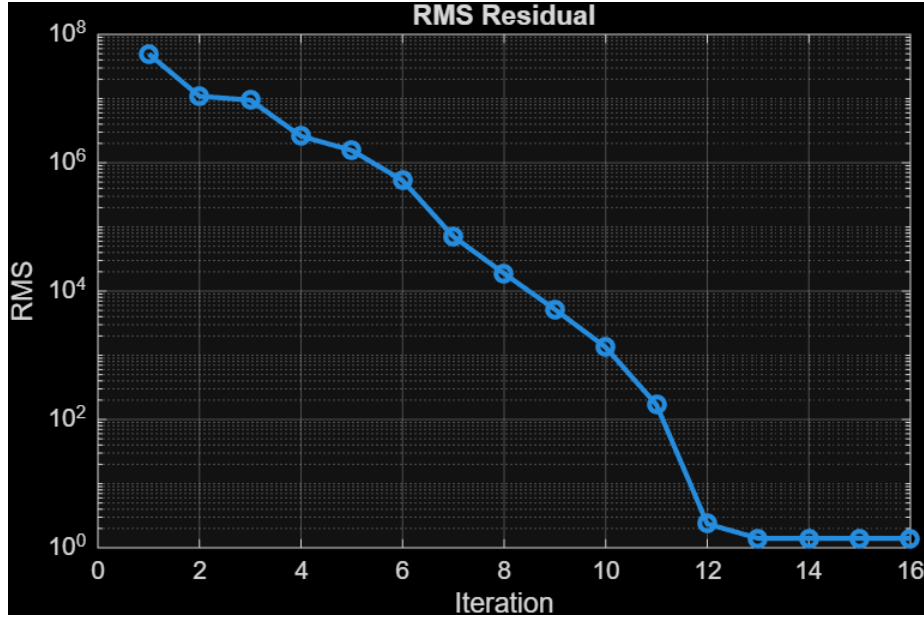


Figure 12: BLSLS RMS Residual with High Deviation in Initial State

In the first criteria, BLS with line search does better than basic BLS. The benefit of using line search is very apparent, as the BLS struggled to even do 50 m and 5 m/s deviation, whereas the BLS with line search was easily able to converge with deviations of 1000 m and 100 m/s. The drawback with the BLSLS is that it takes more iterations even for the low and medium tests, so its real time application is hurt a little. However, both of these filters converge in under 20 s for all tests, so the increased compute time is very marginal for the benefits it gives.

4.2.3 UKF

For the UKF, the results will be shown only for the high deviation test for brevity, as there is a plot per state for each of the tests.

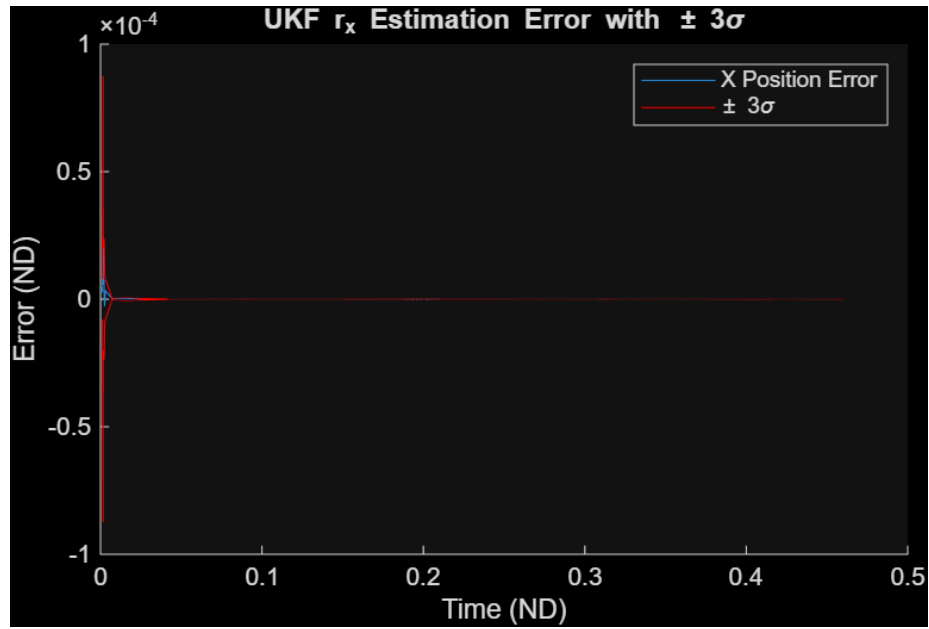


Figure 13: UKF r_x Error with High Initial Deviation in Initial State

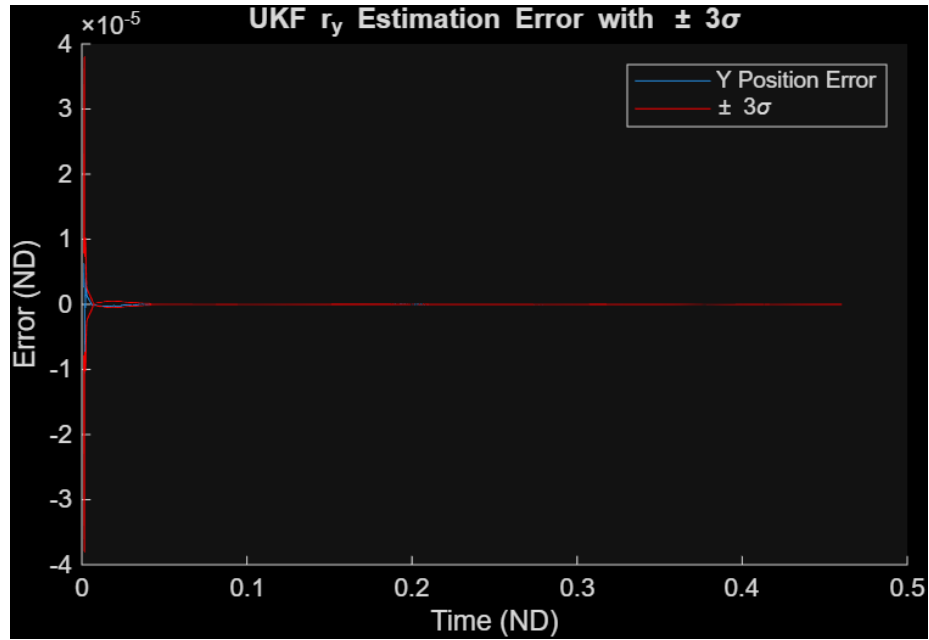


Figure 14: UKF r_y Error with High Initial Deviation in Initial State

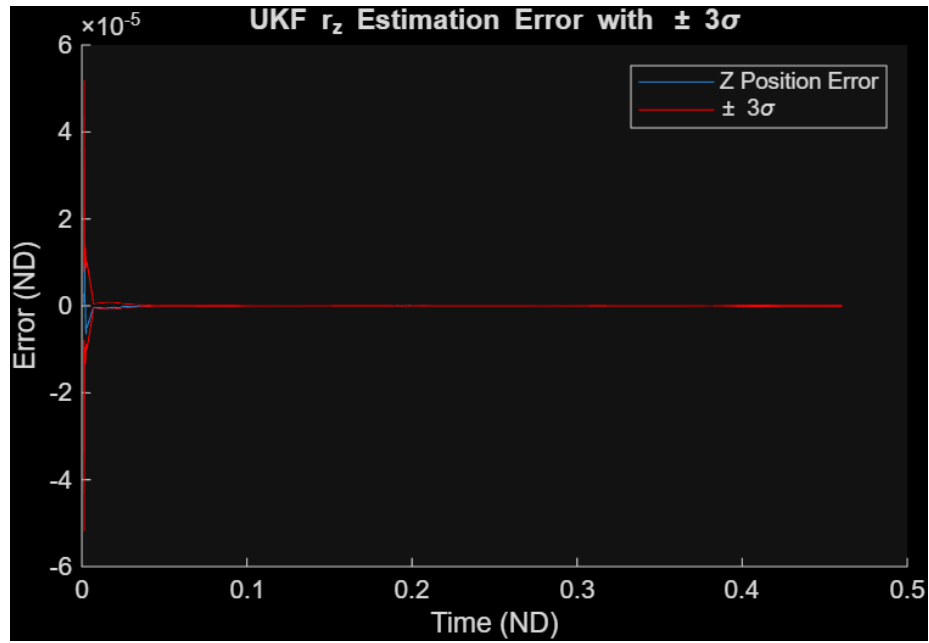


Figure 15: UKF r_z Error with High Initial Deviation in Initial State

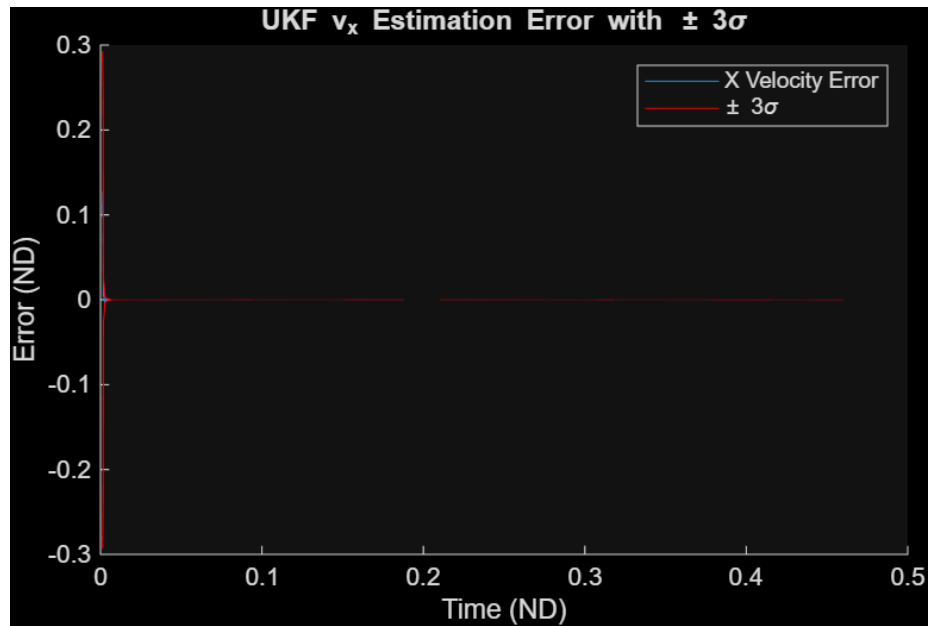


Figure 16: UKF v_x Error with High Initial Deviation in Initial State

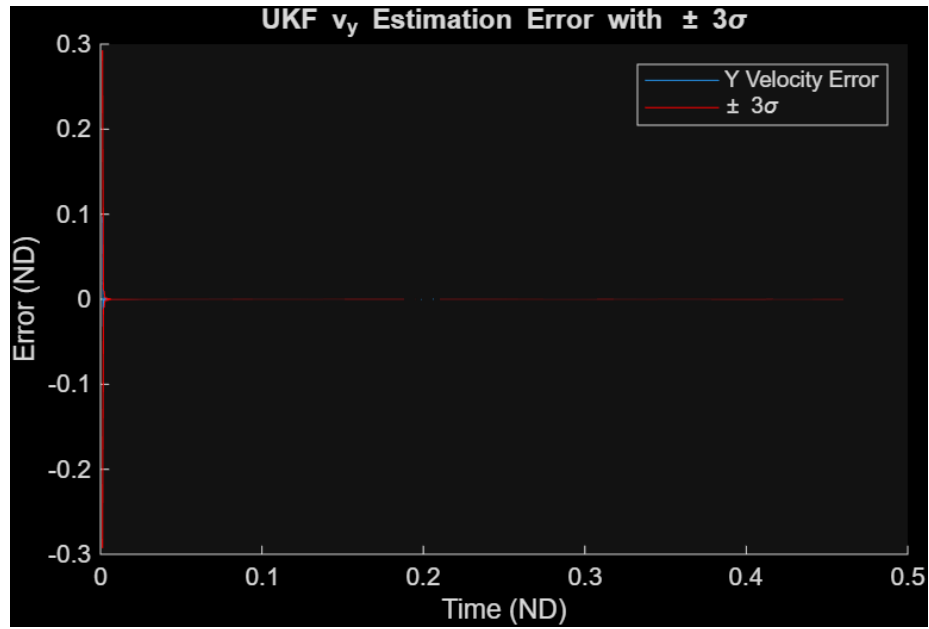


Figure 17: UKF v_y Error with High Initial Deviation in Initial State

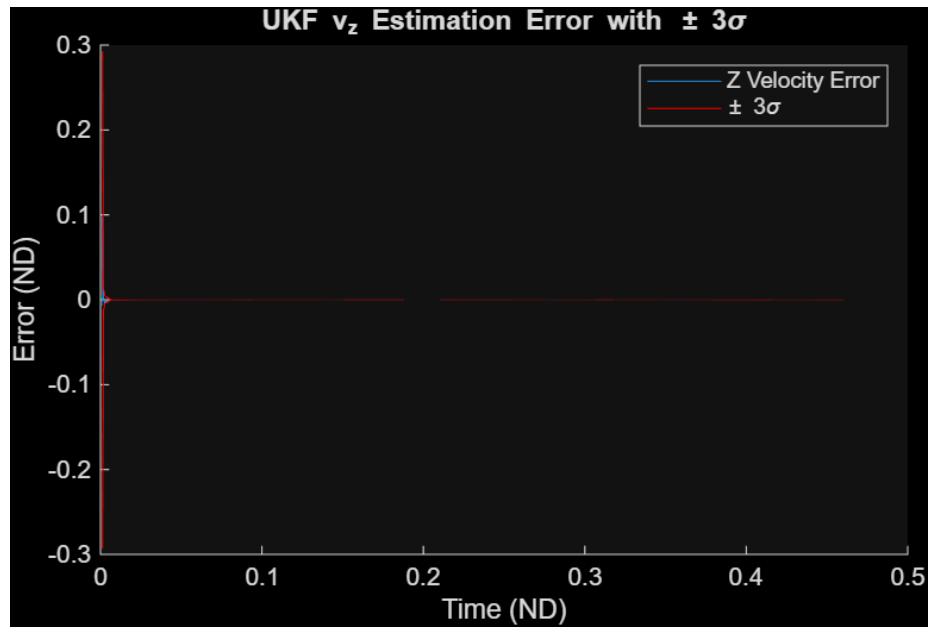


Figure 18: UKF v_z Error with High Initial Deviation in Initial State

The UKF is the most resilient of the three filters for the first condition. As visible in all of the plots, the UKF is able to very rapidly minimize its state estimation error and uncertainty despite very high deviations in the initial state estimate and being a non iterative method. The UKF had a worse final state estimate than the BLSLS in the high deviation test (11.3 m & 0.0004 m/s vs 3.04 m & 0.0001 m/s); however, the difference is extremely minute, and the UKF finished in half the time. As a result, the UKF is the best performing filter overall for the initial state estimate deviation criteria.

4.3 Measurement Outages

4.3.1 BLS without Line Search

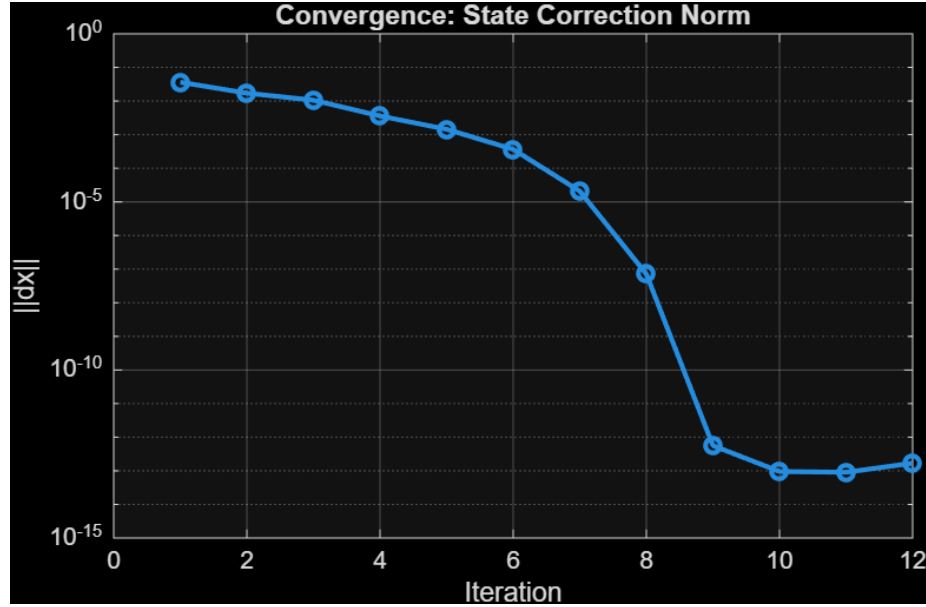


Figure 19: BLS Convergence with $[0.003, 0.44]$ ND Measurement Outage

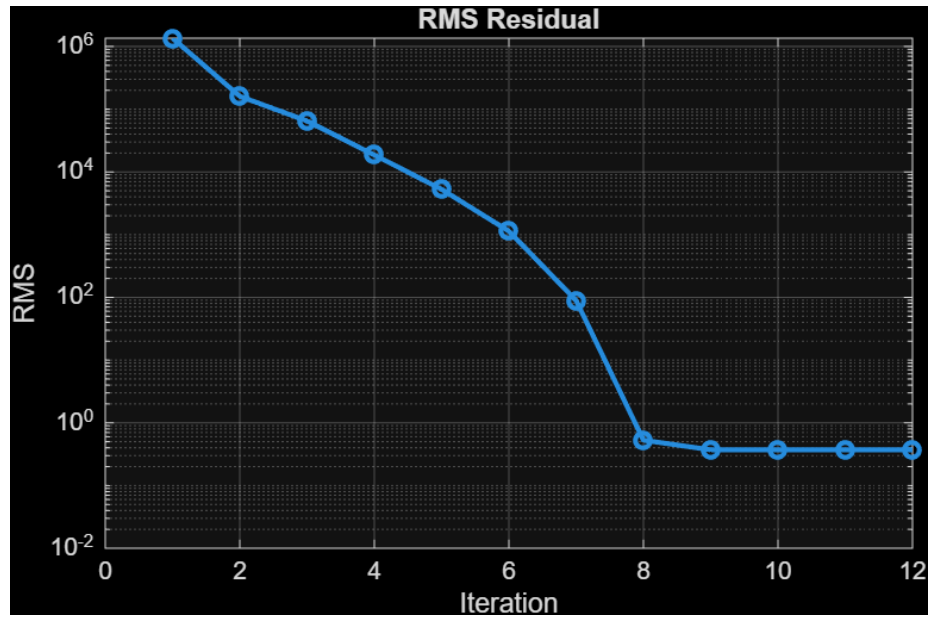


Figure 20: BLS RMS with [0.003, 0.44] ND Measurement Outage

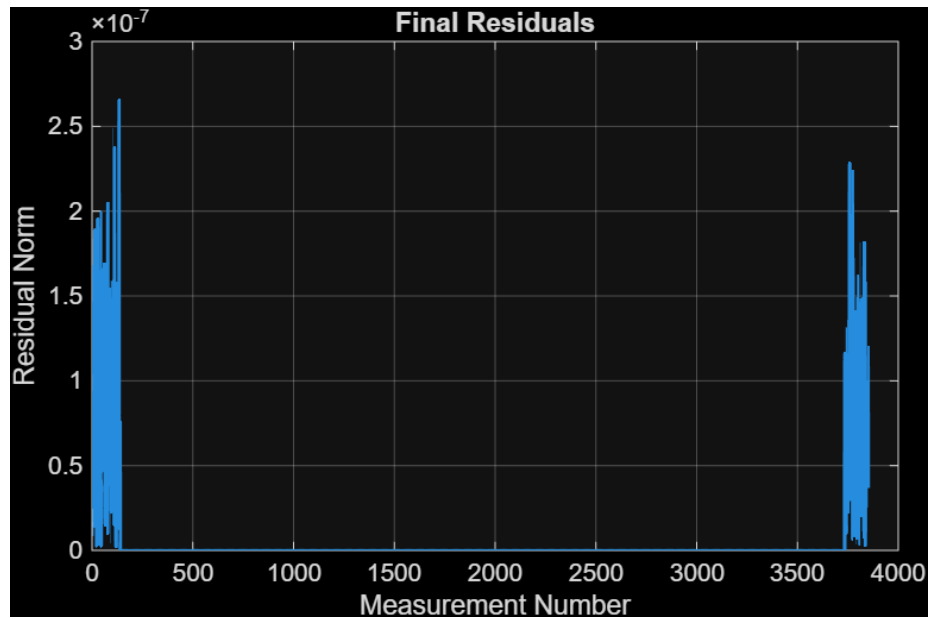


Figure 21: BLS Final Residuals with [0.003, 0.44] ND Measurement Outage

4.3.2 BLS with Line Search

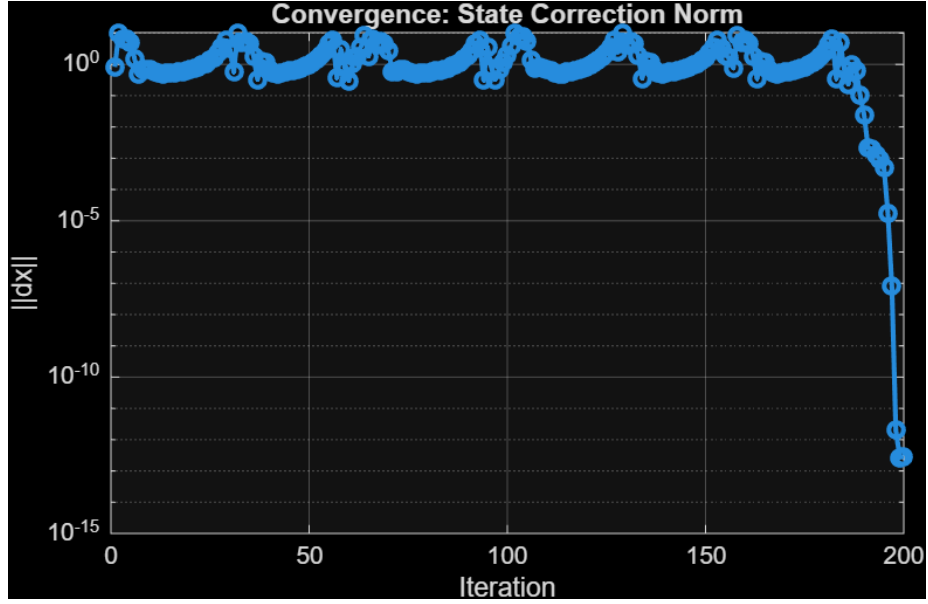


Figure 22: BLSLS Convergence with $[0.003, 0.43]$ ND Measurement Outage

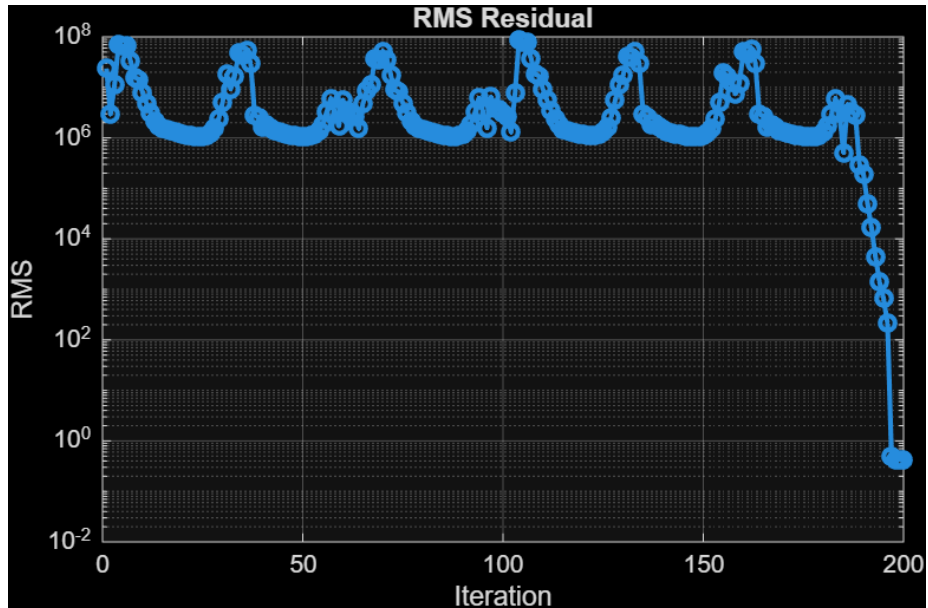


Figure 23: BLSLS RMS with $[0.003, 0.43]$ ND Measurement Outage

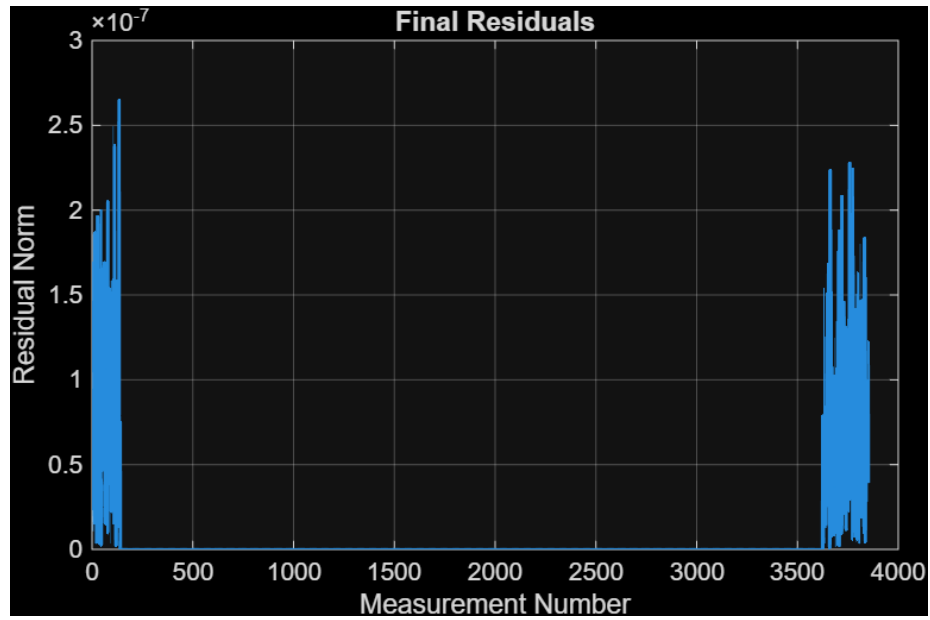


Figure 24: BLSLS Final Residuals with $[0.003, 0.43]$ ND Measurement Outage

4.3.3 UKF

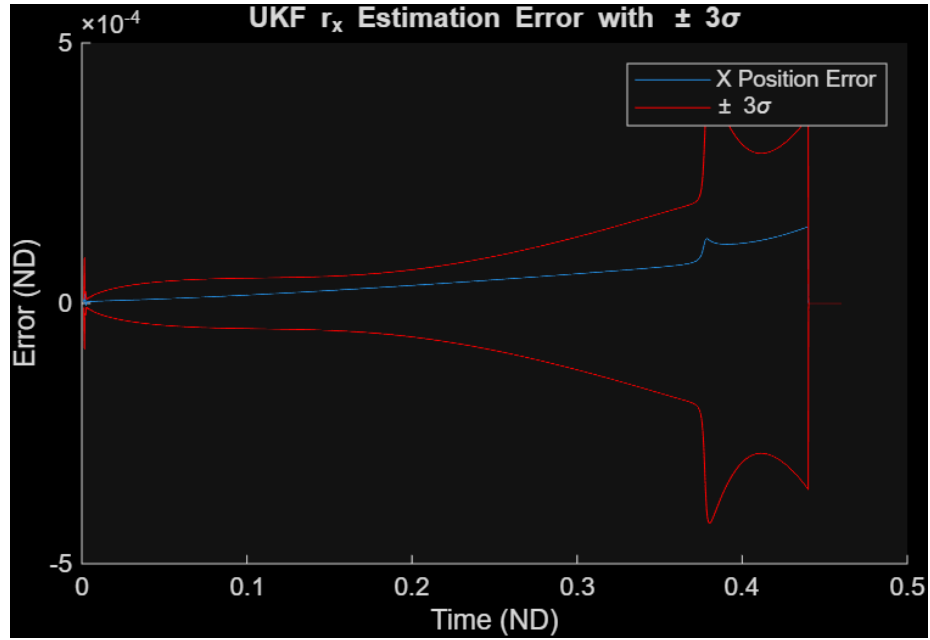


Figure 25: UKF r_x Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

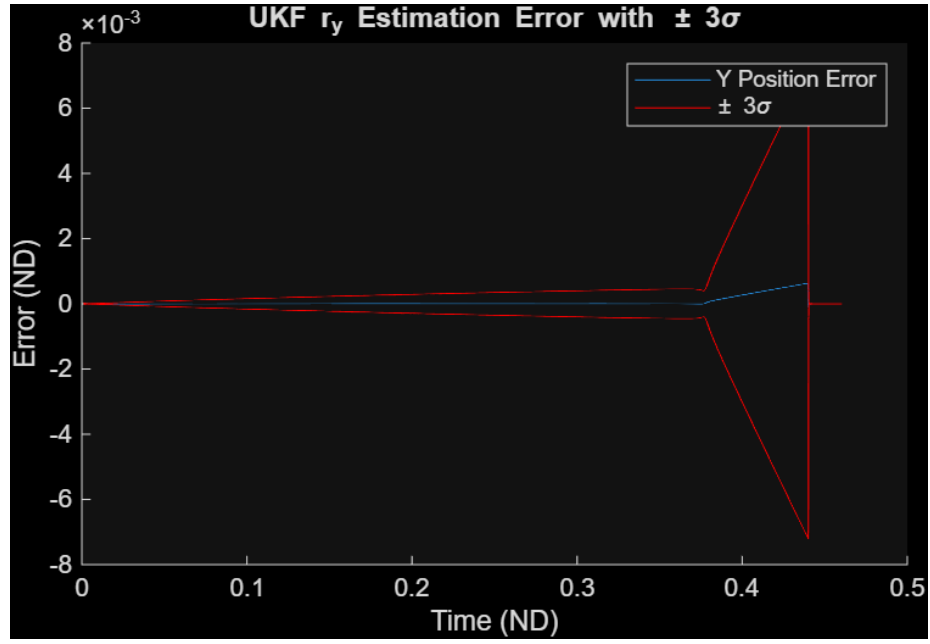


Figure 26: UKF r_y Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

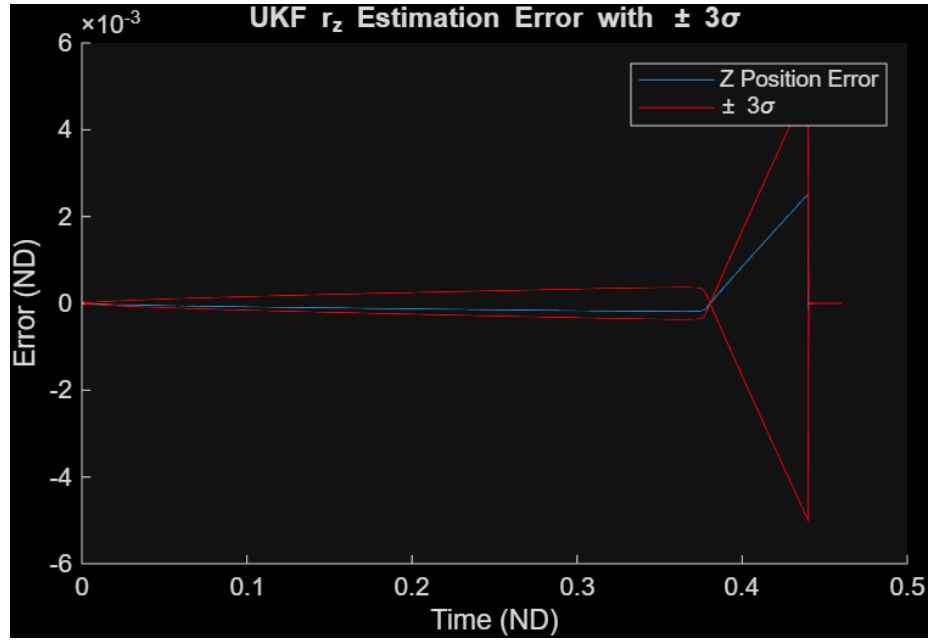


Figure 27: UKF r_z Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

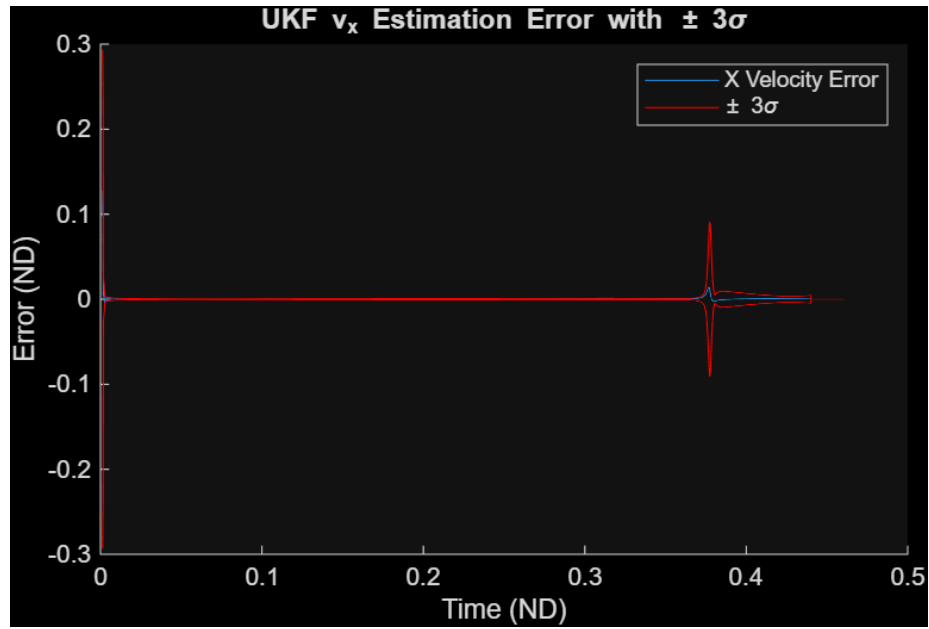


Figure 28: UKF v_x Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

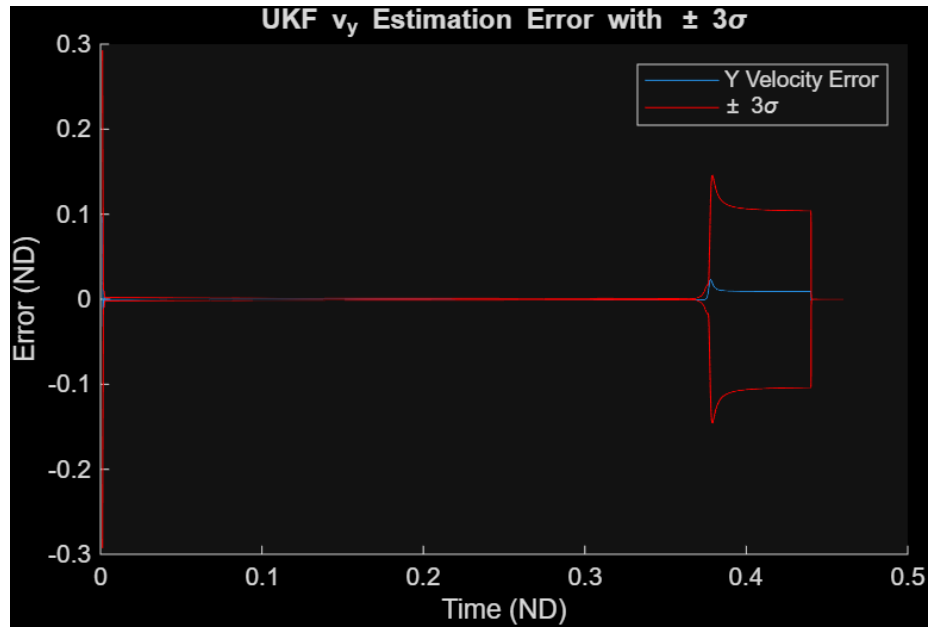


Figure 29: UKF v_y Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

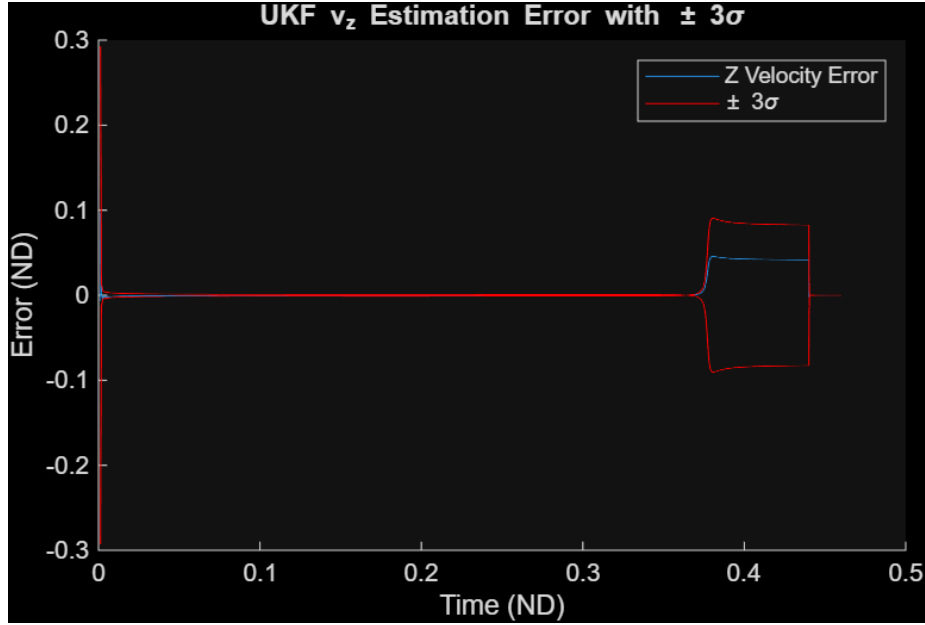


Figure 30: UKF v_z Estimation Error with $[0.003, 0.44]$ ND Measurement Outage

4.3.4 Discussion

The measurement outage test has very interesting results, as it reveals that the most important data is at the beginning, the highly nonlinear areas, and the end. The trajectory simulation used for all of the tests has a gravity assist around the moon before the spacecraft slings out into cislunar space. This event happens at around the 38 hour mark in the trajectory, and the corresponding fallout reveals itself in the latest time measurements can be removed: 0.43/0.44 ND (44.8/45.8 hrs). This limit is very strict, and exceeding 0.44 ND causes all of the filters to diverge and have state estimation errors on the order of millions of meters and thousands of m/s. Consequently, the filters are tested with this upper bound and a lower bound of 0.003 ND, which similarly breaks the filters if gone below. Overall, a very significant amount of data is removed for all of the filters, but they are able to converge as long as the boundaries on the outage were held.

The BLS was tested using $[0.003, 0.44]$ measurement outage domain in the medium initial state deviation condition. It took the filter more iterations to finish, but it was successful. In comparison, running the BLSLS on the $[0.003, 0.44]$ outage domain (high initial state deviation condition) caused the filter to completely diverge. Consequently, it was run instead on the $[0.003, 0.43]$ outage domain. The BLSLS was just barely able to converge, but it required 200 iterations, and the filter would have rapidly broken itself without line search, as seen in the oscillation in Fig. 22 and Fig. 23. The UKF filter was also tested

on the $[0.003, 0.44]$ measurement outage domain, like the BLS, but it was also running in the high initial state deviation condition. Unlike the BLSLS, the UKF did not break at 0.44 ND, but its state estimation error is somewhat high at 1300 m and 0.015 m/s. However, this does not mean the UKF is worse in the smaller outage. Running the UKF in the 0.43 ND scenario results in state estimation errors 98 m and 0.0014 m/s. These errors are higher than the BLSLS in the 0.43 ND scenario, but they are of similar order. Additionally, the biggest benefit is that the UKF ran in 1/4th the time (6 s vs 24 s). The difference in state estimation error is small, so the significantly faster runtime of the UKF makes it the best performing filter once again.

5 Conclusion

This project set out to evaluate whether Batch Least Squares or Unscented Kalman Filtering would be better for highly nonlinear problems, such as Trans-Lunar coast. This comparison came out with a very clear winner: UKF. While BLS has been historically used and was successful in accurate estimation of the spacecraft, the robustness of the UKF is unmatched. As a result, a modified BLS was created using a basic line search method. The BLSLS was a strong upgrade to the BLS, and brought it up to parity with the UKF. It had lower state estimation errors, but the marginal difference ultimately couldn't outweigh the reduced compute time of the UKF. Overall, the BLSLS was the best for state estimation and minimizing error, but the compute speed of the UKF makes it better for any real-time applications.

6 Future Work

This project goes through the different methods and evaluates them rigorously; however, there are some improvements that can be made. First, the CR3BP model, while great for this application, lacks some fidelity. There are methods to improve the accuracy of the model, such as adding in solar perturbations, uneven gravitational fields, etc. Implementing these methods would allow for a more true to life analysis of the methods, as they would be using far more accurate simulation data. Second, this project uses a very basic line search method for the BLSLS. It was implemented very quickly and without implementing any optimality. Rigorously selecting the best line search algorithm could allow for the BLSLS to far exceed its current capabilities, especially in tests like the measurement outage, where it took 200 iterations and barely held on. Between these two upgrades, the second is what would give the most immediate benefit. While the CR3BP fidelity upgrade would help evaluate the filters with more realistic trajectories, the existing model is still very useful in its current state. In comparison, the line search algorithm optimization could result in BLSLS exceeding the UKF, which would be a significant change.

In addition to the upgrades to the current methods, a particle filter would be

interesting to evaluate how many particles are needed to match/exceed the BLSLS/UKF while keeping compute times in a similar region. This problem would be more of an analysis of particle selection, but could be useful for a simple filter implementation.

A CR3BP.py

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.io import savemat
import matplotlib.pyplot as plt
from typing import Tuple, Callable
```

A.1 CR3BP Class Definition

```
class CR3BP:
    def __init__(self):

        """
        Initialize CR3BP model
        """

        # Physical constants
        self.M_earth = 5.9723e24 # kg
        self.M_moon = 7.346e22 # kg
        self.G = 6.674e-11 # m^3/(kg*s^2)
        self.r_em = 384400000 # m

        # Scale factors and nondimensionalization
        # ex. r_dim = r_em * r_nondim
        # ex. t_dim = (TD / 2pi) * t_nondim = TU *
        # t_nondim
        # ex. v_dim = (2pi * r_em) / TD * v_nondim = r_em
        # / TU * v_nondim

        # Total mass, M, and nondimensional mass factor mu
        self.M = self.M_earth + self.M_moon
        self.mu = self.M_moon / self.M
        self.mu_1 = 1 - self.mu
        self.mu_2 = self.mu

        # Time unit, TU, and dimensional physical period TD
        self.TU = np.sqrt(self.r_em ** 3 / (self.G *
            self.M)) # s
        self.TD = self.TU * 2 * np.pi
            # s
        self.TU_days = self.TU / 3600 / 24 # days; ~4.334
            days
        self.TD_days = self.TD / 3600 / 24 # days; ~
            27.285 days

        # Velocity scale factor unit VU
        self.VU = self.r_em / self.TU # m/s
```

```

def Jacobi_Integral(self, state:np.ndarray,
get_U_bar:bool = False) -> float:
    """
    Computes the Jacobi_integral in the rotating frame
    for a spacecraft.
    Args:
        state: [x, y, z, vx, vy, vz] in nondimensional
            units
    Returns:
        Jacobi Integral/Constant, C\n
        Effective Potential U_bar if velocity
            components = 0

    If vx, vy, and vz in state are zero, computes the
    effective potential, U_bar, instead.
    """
    # Collect the state values into position and
    # velocity
    x, y, z = state[0], state[1], state[2]
    vx, vy, vz = state[3], state[4], state[5]

    # Compute r_1 and r_2
    r_1 = np.sqrt((x + self.mu)**2 + y**2 + z**2)
    r_2 = np.sqrt((x - 1 + self.mu)**2 + y**2 + z**2)

    # Compute C
    U_bar = - 1/2 * (x**2 + y**2) - self.mu_1 / r_1 -
        self.mu_2 / r_2 - 1/2 * self.mu_1 * self.mu_2
    V_component = (vx**2 + vy**2 + vz**2)
    C = - V_component - 2 * U_bar

    if get_U_bar:
        return U_bar
    return C

def effective_potentialpartials(self, state):
    """
    Calculate the effective potential U_bar

    Parameters:
        state : current x, y, and z position

    Returns:
        U_bar in x, y, and z
    """
    x, y, z = state[0], state[1], state[2]
    U_bar_x = (self.mu_1*(2*self.mu_2 +
        2*x))/(2*((self.mu_2 + x)**2 + y**2 +
        z**2)**(3/2)) - (self.mu_2*(2*self.mu_1 -

```

```

        2*x))/(2*((self.mu_1 - x)**2 + y**2 +
        z**2)**(3/2)) - x
    U_bar_y = (self.mu_1*y)/((self.mu_2 + x)**2 + y**2
        + z**2)**(3/2) - y + (self.mu_2*y)/((self.mu_1 -
        x)**2 + y**2 + z**2)**(3/2)
    U_bar_z = (self.mu_1*z)/((self.mu_2 + x)**2 + y**2
        + z**2)**(3/2) + (self.mu_2*z)/((self.mu_1 -
        x)**2 + y**2 + z**2)**(3/2)

    return U_bar_x, U_bar_y, U_bar_z

def dynamics(self, t: float, state: np.ndarray) ->
    np.ndarray:
    """
    Dynamics of spacecraft in CR3BP

    Parameters:
        t : time
        state : stacked position and velocity state

    Returns:
        Stacked vector of velocity and acceleration
    """

    # Collect the velocity values
    vx, vy, vz = state[3], state[4], state[5]
    v = state[3:]

    U_bar = self.Jacobi_Integral(state)
    # Get partials
    U_bar_x, U_bar_y, U_bar_z =
        self.effective_potential_partials(state)

    # Calculate acceleration terms
    ax = 2 * vy - U_bar_x
    ay = -2 * vx - U_bar_y
    az = -U_bar_z
    accel = np.array([ax, ay, az])

    return np.concatenate([v, accel])

def integrate_trajectory(self,
    state0: np.ndarray,
    t_span: Tuple[float, float],
    n_points: int = 1000,
    rtol: float = 1e-11,
    atol: float = 1e-14,
    DOP853: bool = False) -> dict:
    """

```

```

Integrate trajectory using adaptive step-size
method.

Parameters:
    state0: Initial state [x, y, z, vx, vy, vz]
    t_span: (t_start, t_end) in non-dimensional
            time units
    n_points: Approximate number of output points
    rtol: Relative tolerance for integrator
    atol: Absolute tolerance for integrator
    DOP853: Use DOP853 integrator

Returns:
    Dictionary with 't', 'y' (states), and 'r'
    (positions)
"""

# Time vector for solution output
t_eval = np.linspace(t_span[0], t_span[1], n_points)
integrator = 'RK45'
if DOP853:
    integrator = 'DOP853'

solution = solve_ivp(
    self.dynamics,
    t_span=t_span,
    y0=state0,
    t_eval=t_eval,
    method=integrator,
    rtol=rtol,
    atol=atol,
    dense_output=True
)

# Extract position vectors
positions = solution.y[:3, :] # Shape: (3,
                               n_points)

return {
    't': solution.t,
    'y': solution.y,
    'r': positions,
    'status': solution.status,
    'message': solution.message
}

```

A.2 LunarTransferTrajectory Class

```

class LunarTransferTrajectory:
    """Generate realistic lunar transfer trajectories."""

    def __init__(self, cr3bp: CR3BP):
        """
        Args:
            cr3bp: CR3BP instance
        """
        self.cr3bp = cr3bp
        self.mu = cr3bp.mu
        self.mu_1 = cr3bp.mu_1
        self.mu_2 = cr3bp.mu_2

    def create_leo_initial_state(self,
                                altitude_km: float = 200,
                                inclination_deg: float =
                                    28.5) -> np.ndarray:
        """
        Create initial state from Low Earth Orbit (LEO).

        Args:
            altitude_km: Altitude above Earth surface (km)
            inclination_deg: Orbital inclination (degrees)

        Returns:
            State vector in non-dimensional CR3BP units
        """
        # Earth radius
        R_earth_m = 6.371e6 # m

        # Orbital radius in meters
        r_leo_m = R_earth_m + altitude_km * 1e3 # m

        # Orbital radius in non-dimensional units
        r_leo_nd = r_leo_m / self.cr3bp.r_em

        # Circular orbit velocity in SI units: v =
            sqrt(GM_earth / r)
        v_leo_si = np.sqrt(self.cr3bp.G *
                            self.cr3bp.M_earth / r_leo_m)

        # Convert to non-dimensional units
        v_leo_nd = v_leo_si / self.cr3bp.VU

        # Initial position in inertial frame (Earth at
            -mu_2)
        # For simplicity, start at periapsis on equator
        x0 = -self.mu_2 - r_leo_nd
        y0 = 0.0
        z0 = 0.0

```

```

# Velocity (circular orbit, tangential to radius)
vx0 = 0.0
vy0 = v_leo_nd
vz0 = 0.0

state = np.array([x0, y0, z0, vx0, vy0, vz0])

print(f"\nLEO Initial Conditions:")
print(f"  Position (ND): [{x0:.6f}, {y0:.6f},
    {z0:.6f}]" )
print(f"  Velocity (ND): [{vx0:.6f}, {vy0:.6f},
    {vz0:.6f}]" )
print(f"  Distance from Earth (m): {r_leo_m:.1f}" )
print(f"  Distance from Earth (ND): {r_leo_nd:.6f}" )
print(f"  Orbital velocity (SI): {v_leo_si:.2f}
    m/s" )
print(f"  Orbital velocity (ND): {v_leo_nd:.6f}" )

return state

def create_trans_lunar_injection(self, leo_state:
np.ndarray) -> np.ndarray:
    """
    Apply Trans-Lunar Injection (TLI) burn to LEO state.

    Increases velocity tangentially to escape Earth and
    reach Moon.

    Args:
        leo_state: LEO state vector

    Returns:
        State after TLI burn
    """

    delta_v_si = 3.345 * 1000 # m/s
    delta_v_nd = delta_v_si / self.cr3bp.VU

    tli_state = leo_state.copy()
    tli_state[4] += delta_v_nd # Increase velocity in
        y-direction

    v_leo = np.linalg.norm(leo_state[3:6])
    v_tli = np.linalg.norm(tli_state[3:6])

    print(f"\nTLI Maneuver:")
    print(f"  Delta V requested: {delta_v_si/1000:.2f}
        km/s = {delta_v_si:.1f} m/s" )
    print(f"  Delta V (ND): {delta_v_nd:.6f}" )

```

```

print(f"   Velocity before TLI (ND): {v_leo:.6f}
      ({v_leo * self.cr3bp.VU:.2f} m/s)")
print(f"   Velocity after TLI (ND): {v_tli:.6f}
      ({v_tli * self.cr3bp.VU:.2f} m/s)")

return tli_state

def generate_trajectory(self,
                        initial_state: np.ndarray,
                        duration_days: float = 5,
                        n_points: int = 10000) -> dict:
    """
    Generate full lunar transfer trajectory.

    Args:
        initial_state: State vector [x, y, z, vx, vy,
                                   vz]
        duration_days: Duration in days
        n_points: Number of trajectory points

    Returns:
        Trajectory dictionary
    """
    # Convert duration to non-dimensional time
    time_unit_days = self.cr3bp.TD_days / (2 * np.pi)
    duration_nd = duration_days / time_unit_days

    print(f"\nTrajectory Generation:")
    print(f"   Duration: {duration_days} days
          ({duration_nd:.4f} ND time units)")
    print(f"   1 ND time unit = {time_unit_days:.4f}
          days")

    # Integrate
    trajectory = self.cr3bp.integrate_trajectory(
        initial_state,
        t_span=(0, duration_nd),
        n_points=n_points
    )

    # Convert time back to days
    trajectory['t_days'] = trajectory['t'] *
        time_unit_days

    print(f"   Status: {trajectory['status']} -
          {trajectory['message']}")
    print(f"   Generated {len(trajectory['t'])}
          trajectory points")

    return trajectory

```

A.3 analyze_trajectory() Function

```
def analyze_trajectory(trajectory: dict, cr3bp: CR3BP) ->
None:
    """
    Analyze trajectory for physical correctness.

    Args:
        trajectory: Trajectory dictionary from
            generate_trajectory
        cr3bp: CR3BP instance
    """
    print("\n" + "="*60)
    print("TRAJECTORY ANALYSIS")
    print("="*60)

    t = trajectory['t_days']
    r = trajectory['r']
    v = trajectory['y'][3:6]

    # Compute distances and speeds
    r_earth = np.sqrt((r[0] + cr3bp.mu_2)**2 + r[1]**2 +
        r[2]**2)
    r_moon = np.sqrt((r[0] - cr3bp.mu_1)**2 + r[1]**2 +
        r[2]**2)
    speed_nd = np.linalg.norm(v, axis=0)
    speed_si = speed_nd * cr3bp.VU

    # Convert distances to km
    r_earth_km = r_earth * cr3bp.r_em / 1e3
    r_moon_km = r_moon * cr3bp.r_em / 1e3

    # Initial and final conditions
    print(f"\nInitial State (t = {t[0]:.3f} days):")
    print(f"    Distance to Earth: {r_earth[0]:.4f} ND
        ({r_earth_km[0]:.1f} km)")
    print(f"    Distance to Moon: {r_moon[0]:.4f} ND
        ({r_moon_km[0]:.1f} km)")
    print(f"    Speed: {speed_nd[0]:.6f} ND
        ({speed_si[0]:.2f} m/s = {speed_si[0]/1000:.2f}
        km/s)")

    print(f"\nFinal State (t = {t[-1]:.3f} days):")
    print(f"    Distance to Earth: {r_earth[-1]:.4f} ND
        ({r_earth_km[-1]:.1f} km)")
    print(f"    Distance to Moon: {r_moon[-1]:.4f} ND
        ({r_moon_km[-1]:.1f} km)")
```



```

print(f"   Speed: {speed_nd[-1]:.6f} ND
      ({speed_si[-1]:.2f} m/s = {speed_si[-1]/1000:.2f}
      km/s)")

# Min/max distances
print(f"\nDistance Statistics:")
print(f"   Min distance to Earth: {np.min(r_earth):.4f}
      ND ({np.min(r_earth_km):.1f} km)")
print(f"   Max distance to Earth: {np.max(r_earth):.4f}
      ND ({np.max(r_earth_km):.1f} km)")
print(f"   Min distance to Moon: {np.min(r_moon):.4f}
      ND ({np.min(r_moon_km):.1f} km)")
print(f"   Max distance to Moon: {np.max(r_moon):.4f}
      ND ({np.max(r_moon_km):.1f} km)")

# Speed statistics
print(f"\nVelocity Statistics:")
print(f"   Initial speed: {speed_nd[0]:.6f} ND
      ({speed_si[0]:.2f} m/s)")
print(f"   Max speed: {np.max(speed_nd):.6f} ND
      ({np.max(speed_si):.2f} m/s)")
print(f"   Min speed: {np.min(speed_nd):.6f} ND
      ({np.min(speed_si):.2f} m/s)")

# Transition point (where Moon distance ~ Earth
# distance)
crossover_idx = np.argmin(np.abs(r_earth - r_moon))
print(f"\nGravitational Transition:")
print(f"   Earth-Moon crossover at t =
      {t[crossover_idx]:.3f} days")
print(f"   Distance at crossover:
      {r_earth[crossover_idx]:.4f} ND
      ({r_earth_km[crossover_idx]:.1f} km)")
print(f"   Speed at crossover:
      {speed_nd[crossover_idx]:.6f} ND
      ({speed_si[crossover_idx]:.2f} m/s)")

# Jacobi constant check
energy = []
for i in range(len(t)):
    state = trajectory['y'][:, i]
    U = cr3bp.Jacobi_Integral(state)
    v_squared = np.sum(v[:, i]**2)
    C = cr3bp.Jacobi_Integral(state)
    energy.append(C)
energy = np.array(energy)

print(f"\nEnergy Conservation:")
print(f"   Jacobi constant (initial): {energy[0]:.4f}")
print(f"   Jacobi constant (final): {energy[-1]:.4f}")

```

```

print(f"    Variation: {np.max(energy) -
      np.min(energy):.6f}")
print(f"    Relative variation: {(np.max(energy) -
      np.min(energy)) / np.abs(energy[0]) * 100:.4f}%")

if (np.max(energy) - np.min(energy)) /
    np.abs(energy[0]) > 0.01:
    print("        WARNING: Energy variation > 1%, check
          integration accuracy")
else:
    print("        Energy is well-conserved")

print("="*60 + "\n")

```

A.4 plot_trajectory() Function

```

def plot_trajectory(trajecory: dict, cr3bp: CR3BP,
                   save_path: str = None):
    """
    Plot lunar transfer trajectory in rotating frame.

    Args:
        trajectory: Trajectory dictionary from
                    generate_trajectory
        cr3bp: CR3BP instance
        save_path: Optional path to save figure
    """
    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    # Plot 1: XY plane view
    ax = axes[0, 0]
    ax.plot(trajectory['r'][0], trajectory['r'][1], 'b-',
            linewidth=1, label='Spacecraft')
    ax.plot(-cr3bp.mu_2, 0, 'go', markersize=10,
            label='Earth')
    ax.plot(cr3bp.mu_1, 0, 'ks', markersize=8, label='Moon')
    ax.set_xlabel('X (ND)')
    ax.set_ylabel('Y (ND)')
    ax.set_title('Lunar Transfer Trajectory - XY Plane')
    ax.grid(True, alpha=0.3)
    ax.axis('equal')
    ax.legend()

    # Plot 2: Distance from Earth and Moon
    ax = axes[0, 1]
    pos = trajectory['r']
    r_earth = np.sqrt((pos[0] + cr3bp.mu_2)**2 + pos[1]**2
                     + pos[2]**2)

```

```

r_moon = np.sqrt((pos[0] - cr3bp.mu_1)**2 + pos[1]**2 +
pos[2]**2)

# Convert to km for plotting
r_earth_km = r_earth * cr3bp.r_em / 1e3
r_moon_km = r_moon * cr3bp.r_em / 1e3

ax.semilogy(trajecory['t_days'], r_earth_km, 'g-',
label='Distance to Earth', linewidth=2)
ax.semilogy(trajecory['t_days'], r_moon_km, 'k-',
label='Distance to Moon', linewidth=2)
ax.set_xlabel('Time (days)')
ax.set_ylabel('Distance (km)')
ax.set_title('Distance to Primaries')
ax.grid(True, alpha=0.3, which='both')
ax.legend()

# Plot 3: Speed profile
ax = axes[1, 0]
velocity = trajectory['y'][3:6]
speed_nd = np.linalg.norm(velocity, axis=0)
speed_si = speed_nd * cr3bp.VU
ax.plot(trajectory['t_days'], speed_si/1000, 'r-',
linewidth=2)
ax.set_xlabel('Time (days)')
ax.set_ylabel('Speed (km/s)')
ax.set_title('Spacecraft Speed')
ax.grid(True, alpha=0.3)

# Plot 4: Energy (Jacobi Integral)
ax = axes[1, 1]
energy = []
for i in range(len(trajectory['t'])):
    state = trajectory['y'][:, i]
    # Jacobi constant (energy-like quantity in CR3BP)
    U = cr3bp.Jacobi_Integral(state)
    v_squared = np.sum(state[3:6]**2)
    C = cr3bp.Jacobi_Integral(state)
    energy.append(C)
ax.plot(trajectory['t_days'], energy, 'm-', linewidth=1)
ax.set_xlabel('Time (days)')
ax.set_ylabel('Jacobi Constant C')
ax.set_title('Energy Conservation Check')
ax.grid(True, alpha=0.3)

plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=150)
plt.show()

```

A.5 main() Script

```
def main():
    print("="*60)
    print("AERO 626 - CR3BP Implementation")
    print("Lunar Transfer Trajectory Generation")
    print("="*60)

    # Initialize CR3BP
    cr3bp = CR3BP()

    # Create trajectory generator
    traj_gen = LunarTransferTrajectory(cr3bp)

    # Generate LEO initial state
    leo_state =
        traj_gen.create_leo_initial_state(altitude_km=200)

    # Apply TLI burn
    tli_state =
        traj_gen.create_trans_lunar_injection(leo_state)

    # Generate trajectory
    trajectory = traj_gen.generate_trajectory(
        tli_state,
        duration_days=2.0,
        n_points=10000
    )

    # Analyze trajectory
    analyze_trajectory(trajectory, cr3bp)

    # Plot results
    print("\nGenerating plots...")
    plot_trajectory(trajectory, cr3bp, save_path=None)

    # Save trajectory for later use
    np.save('trajectory_true.npy', {
        't': trajectory['t'],
        'y': trajectory['y'],
        't_days': trajectory['t_days']
    }, allow_pickle=True)
    savemat('trajectory_true.mat', {
        't': trajectory['t'],
        'y': trajectory['y'],
        't_days': trajectory['t_days']
    })

    print("\nTrajectory saved to trajectory_true.npy")
```

```
        return trajectory, cr3bp

if __name__ == '__main__':
    trajectory, cr3bp = main()
```

B measurement_model.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import savemat
from dataclasses import dataclass
from typing import List, Tuple, Optional
```

B.1 GroundStation Dataclass

```
@dataclass
class GroundStation:
    """Ground station with geodetic coordinates"""
    name: str
    latitude: float # degrees
    longitude: float # degrees
    altitude: float # meters above sea level
```

B.2 MeasurementSimulator Class

```
class MeasurementSimulator:
    """Simulate range and Doppler measurements from ground
    stations"""

    def __init__(self, stations: List[GroundStation],
                  sigma_range: float = 1.0, # meters
                  sigma_range_rate: float = 1e-4, # m/s
                  earth_radius: float = 6378137.0, # meters
                  moon_distance: float = 384400000.0, #
                      meters
                  earth_angular_vel: float = 7.2921159e-5):
        # rad/s

        """
        Initialize measurement simulator

        Parameters:
        -----
        stations : List of GroundStation objects
        sigma_range : Range measurement noise std dev
                      (meters)
        sigma_range_rate : Range rate measurement noise std
                          dev (m/s)
        earth_radius : Earth equatorial radius (meters)
        moon_distance : Earth-Moon distance (meters)
        earth_angular_vel : Earth rotation rate (rad/s)
        """
```

```

self.stations = stations
self.sigma_range = sigma_range
self.sigma_range_rate = sigma_range_rate
self.R_earth = earth_radius
self.d_moon = moon_distance
self.omega_earth = earth_angular_vel

# CR3BP parameters for Earth-Moon system
self.mu = 0.012150585609624 # Moon mass ratio

def geodetic_to_ecef(self, lat: float, lon: float, alt:
float) -> np.ndarray:
    """
    Convert geodetic coordinates to ECEF
    (Earth-Centered Earth-Fixed)

    Parameters:
    -----
    lat : latitude in degrees
    lon : longitude in degrees
    alt : altitude in meters

    Returns:
    -----
    position : 3D position in ECEF frame (meters)
    """
    lat_rad = np.deg2rad(lat)
    lon_rad = np.deg2rad(lon)

    # Simplified spherical Earth (for more accuracy,
    use WGS84 ellipsoid)
    r = self.R_earth + alt

    x = r * np.cos(lat_rad) * np.cos(lon_rad)
    y = r * np.cos(lat_rad) * np.sin(lon_rad)
    z = r * np.sin(lat_rad)

    return np.array([x, y, z])

def ecef_to_cr3bp(self, r_ecef: np.ndarray, time:
float) -> np.ndarray:
    """
    Convert ECEF coordinates to CR3BP rotating frame

    Parameters:
    -----
    r_ecef : position in ECEF frame (meters)
    time : time in seconds

    Returns:

```

```

-----
r_cr3bp : position in CR3BP rotating frame
         (normalized units)
"""
# Rotation angle due to Earth's rotation
theta_earth = self.omega_earth * time

# Rotation matrix from ECEF to ECI (inertial)
R_z = np.array([
    [np.cos(theta_earth), -np.sin(theta_earth),
     0.0],
    [np.sin(theta_earth),  np.cos(theta_earth),
     0.0],
    [0,                    0,                    1.0]
])
r_eci = R_z @ r_ecef

# Convert to CR3BP frame (normalized by Earth-Moon
# distance)
t_norm = time / cr3bp.TU
theta_cr3bp = t_norm

c = np.cos(-theta_cr3bp)
s = np.sin(-theta_cr3bp)

R_cr = np.array([[c,  -s,  0.0],
                  [s,   c,  0.0],
                  [0.0, 0.0, 1.0]])

r_rot = R_cr @ r_eci

# In CR3BP, Earth is at [-mu, 0, 0] and Moon is at
# [1-mu, 0, 0]
r_normalized = r_rot / cr3bp.r_em

# Offset so Earth is at proper CR3BP position
r_cr3bp = r_normalized + np.array([-self.mu, 0, 0])

return r_cr3bp

def station_position_velocity(self, station:
    GroundStation,
                                time: float) ->
    Tuple[np.ndarray,
          np.ndarray]:
    """
    Calculate ground station position and velocity in
    CR3BP frame

    Parameters:

```



```

-----
station : GroundStation object
time : time in seconds

Returns:
-----
position : 3D position in CR3BP frame (normalized)
velocity : 3D velocity in CR3BP frame (normalized)
"""
# Get ECEF position
r_ecef = self.geodetic_to_ecef(station.latitude,
                               station.longitude,
                               station.altitude)

# Convert to CR3BP
r_cr3bp = self.ecef_to_cr3bp(r_ecef, time)

# Velocity due to Earth's rotation in ECEF frame
# v = omega x r
omega_vec = np.array([0, 0, self.omega_earth])
v_ecef = np.cross(omega_vec, r_ecef)

# Convert velocity to ECI frame
theta_earth = self.omega_earth * time
R_z = np.array([
    [np.cos(theta_earth), -np.sin(theta_earth), 0],
    [np.sin(theta_earth),  np.cos(theta_earth), 0],
    [0.0,                0.0,                1]
])
v_eci = R_z @ v_ecef

# Convert Velocity to CR3BP frame
t_norm = time / cr3bp.TU
theta_cr3bp = t_norm
c = np.cos(-theta_cr3bp); s = np.sin(-theta_cr3bp)
R_cr = np.array([[c,  -s,  0.0],
                  [s,   c,  0.0],
                  [0.0, 0.0, 1.0]])

v_rot_si = R_cr @ v_eci          # m/s (rotated
                                inertial velo)

# Normalize velocity
# In CR3BP, velocity is normalized by
sqrt(G*(M1+M2)/d)
# For dimensional velocity in m/s, divide by
characteristic velocity

v_rot_nd = v_rot_si / cr3bp.VU
omega_nd = np.array([0.0, 0.0, 1.0])

```

```

v_cr3bp = v_rot_nd - np.cross(omega_nd, r_cr3bp)

return r_cr3bp, v_cr3bp

def is_visible(self, r_spacecraft: np.ndarray,
               r_station: np.ndarray) -> bool:
    """
    Check if spacecraft is visible from ground station
    (above horizon)

    Parameters:
    -----
    r_spacecraft : spacecraft position in CR3BP frame
    r_station : station position in CR3BP frame

    Returns:
    -----
    visible : True if spacecraft is above station
              horizon
    """
    # Vector from station to spacecraft
    los_vector = r_spacecraft - r_station

    # Vector from Earth center to station (in CR3BP,
    # Earth is at [-mu, 0, 0])
    r_earth_center = np.array([-self.mu, 0, 0])
    station_radial = r_station - r_earth_center

    # Check if elevation angle is positive (above
    # horizon)
    # cos(elevation) = dot(los, radial) / (|los| *
    # |radial|)
    cos_angle = np.dot(los_vector, station_radial)
    cos_angle /= (np.linalg.norm(los_vector) *
                  np.linalg.norm(station_radial))

    # If angle < 90 degrees, spacecraft is above horizon
    # Add small margin for atmospheric effects (~5
    # degrees)
    elevation_limit = np.cos(np.deg2rad(95))

    return cos_angle > elevation_limit

def generate_measurement(self, r_spacecraft: np.ndarray,
                        v_spacecraft: np.ndarray,
                        station: GroundStation,
                        time: float,
                        num_measurements: int) ->
    Optional[Tuple[float, float,
                    str]]:

```

```

"""
Generate range and range-rate measurements if
    visible

Parameters:
-----
r_spacecraft : spacecraft position in CR3BP frame
v_spacecraft : spacecraft velocity in CR3BP frame
time : measurement time in seconds
num_measurements : number of measurements taken so
    far

Returns:
-----
(range, range_rate, station_name) or None if not
    visible
"""
# Get station position and velocity
r_station, v_station =
    self.station_position_velocity(station, time)

# Check visibility
if not self.is_visible(r_spacecraft, r_station):
    return None

if num_measurements < 10:
    print("DEBUG station:", station.name, " | |v -
        omega x r| (ND) = ",
        np.linalg.norm(v_station -
            np.cross([0,0,1.0], r_station)))

# Calculate range (convert back to meters for
    measurement)
relative_pos = (r_spacecraft - r_station) *
    cr3bp.r_em
range_true = np.linalg.norm(relative_pos)
range_meas = range_true + np.random.normal(0,
    self.sigma_range)

# Calculate range rate (convert back to m/s)
relative_vel = (v_spacecraft - v_station) * cr3bp.VU

range_rate_true = np.dot(relative_pos,
    relative_vel) / range_true
range_rate_meas = range_rate_true +
    np.random.normal(0, self.sigma_range_rate)

return (range_meas, range_rate_meas, station.name,
    r_station, v_station, range_true,
    range_rate_true)

```

```

def simulate_tracking_pass(self, trajectory: np.ndarray,
                           times: np.ndarray,
                           measurement_interval: float
                           = 60.0) -> List[dict]:
    """
    Simulate full tracking pass over trajectory

    Parameters:
    -----
    trajectory : (N, 6) array of [x, y, z, vx, vy, vz]
                  states in CR3BP
    times : (N,) array of times in seconds
    measurement_interval : measurement cadence in
                          seconds

    Returns:
    -----
    measurements : List of measurement dictionaries
    """
    measurements = []
    last_meas_time = {station.name: -np.inf for station
                      in self.stations}
    num_measurements = 0

    for i, t in enumerate(times):
        r_sc = trajectory[i, :3]
        v_sc = trajectory[i, 3:]

        for station in self.stations:
            # Check if enough time has passed since
            # last measurement
            if t - last_meas_time[station.name] >=
               measurement_interval:
                result =
                    self.generate_measurement(r_sc,
                                              v_sc, station, t, num_measurements)
                if result is not None:
                    # Print x_0_ref
                    if len(measurements) == 0:
                        print(f"Reference Position:
                              {r_sc}")
                        print(f"Reference Velocity:
                              {v_sc}")

                    num_measurements += 1

                range_meas, range_rate_meas,
                station_name, station_pos,
                station_vel, range_true,

```

```

        range_rate_true = result
        measurements.append({
            'time': t,
            'station': station_name,
            'range': range_meas,
            'range_rate': range_rate_meas,
            'range_true': range_true,
            'range_rate_true':
                range_rate_true,
            'spacecraft_pos': r_sc.copy(),
            'spacecraft_vel': v_sc.copy(),
            'station_pos': station_pos,
            'station_vel': station_vel
        })
        last_meas_time[station.name] = t

    return measurements

```

B.3 main() Script

```

if __name__ == "__main__":
    # Import CR3BP code
    from CR3BP import CR3BP, LunarTransferTrajectory

    print("="*60)
    print("GROUND STATION MEASUREMENT SIMULATION")
    print("="*60)

    # Initialize CR3BP and generate trajectory
    print("\n1. Generating CR3BP Lunar Transfer Trajectory")
    cr3bp = CR3BP()
    traj_gen = LunarTransferTrajectory(cr3bp)

    # Generate LEO initial state
    leo_state =
        traj_gen.create_leo_initial_state(altitude_km=200)

    # Apply TLI burn
    tli_state =
        traj_gen.create_trans_lunar_injection(leo_state)

    # Generate trajectory (5 days with high resolution for
    smooth measurements)
    cr3bp_trajectory = traj_gen.generate_trajectory(
        tli_state,
        duration_days=2.0,
        n_points=10000
    )

```

```

# Extract trajectory data
trajectory = cr3bp_trajectory['y'].T # Now shape
      (n_points, 6)

# Convert time from non-dimensional to seconds
times = cr3bp_trajectory['t'] * cr3bp.TU # Convert to
      seconds

print(f"\nTrajectory Info:")
print(f"  Duration: {times[-1]/3600:.2f} hours
      ({times[-1]/(24*3600):.2f} days)")
print(f"  Number of points: {len(times)}")
print(f"  Time step: {np.mean(np.diff(times)):.2f}
      seconds")

# Define Deep Space Network stations
print("\n2. Defining Ground Stations")
stations = [
    GroundStation("Goldstone", 35.426667, -116.89,
                  1000), # California
    GroundStation("Madrid", 40.427222, -4.248889, 800),
    # Spain
    GroundStation("Canberra", -35.401389, 148.981667,
                  600) # Australia
]

# Create simulator with measurement noise
print("\n3. Creating Measurement Simulator")
sim = MeasurementSimulator(
    stations,
    sigma_range=1.0,          # 1 meters range noise
    sigma_range_rate=1e-4,    # 0.001 m/s range rate
    noise,
    moon_distance=cr3bp.r_em # Use Earth-Moon distance
)

# Simulate measurements every 60 seconds
print("\n4. Simulating Tracking Pass")
measurement_interval = 60.0 # s
measurements = sim.simulate_tracking_pass(
    trajectory,
    times,
    measurement_interval=measurement_interval
)

print(f"\n5. Measurement Summary:")
print(f"  Total measurements: {len(measurements)}")
for station in stations:

```

```

        station_count = sum(1 for m in measurements if
                               m['station'] == station.name)
    print(f"    {station.name}: {station_count}
           measurements")

# Calculate measurement statistics
if len(measurements) > 0:
    time_coverage = (measurements[-1]['time'] -
                     measurements[0]['time']) / 3600
    print(f"    Time coverage: {time_coverage:.2f} hours")
    print(f"    Average measurement rate:
           {len(measurements)/time_coverage:.2f} meas/hour")

print(f"\nFirst 10 measurements:")
for i, meas in enumerate(measurements[:10]):
    print(f"    {i+1}. t={meas['time']/3600:.3f} hr,
           {meas['station']:10s}, "
           f"Range={meas['range']/1e6:.3f} Mm, Range
           Rate={meas['range_rate']:7.3f} m/s")

# Save measurements for use in filters
print("\n6. Saving Measurements")
np.save('measurements.npy', measurements,
        allow_pickle=True)
savemat('measurements.mat', {'measurements':
                             measurements})
print("    Saved to 'measurements.npy'")

# Plot measurement availability and coverage
print("\n7. Generating Plots...")
fig = plt.figure(figsize=(16, 10))
gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)

# Plot 1: Range measurements
ax1 = fig.add_subplot(gs[0, 0])
colors = {'Goldstone': 'blue', 'Madrid': 'red',
          'Canberra': 'green'}

for station_name, color in colors.items():
    station_meas = [m for m in measurements if
                     m['station'] == station_name]
    if station_meas:
        times_hrs = [m['time']/3600 for m in
                      station_meas]
        ranges = [m['range']/1000 for m in station_meas]
        ax1.scatter(times_hrs, ranges,
                     label=station_name, color=color, s=3,
                     alpha=0.6)

ax1.set_ylabel('Range (km)', fontsize=11)

```

```

ax1.set_xlabel('Time (hours)', fontsize=11)
ax1.legend(loc='best')
ax1.grid(True, alpha=0.3)
ax1.set_title('Range Measurements', fontsize=12,
              fontweight='bold')

# Plot 2: Range rate measurements
ax2 = fig.add_subplot(gs[0, 1])
for station_name, color in colors.items():
    station_meas = [m for m in measurements if
                    m['station'] == station_name]
    if station_meas:
        times_hrs = [m['time']/3600 for m in
                     station_meas]
        range_rates = [m['range_rate'] for m in
                       station_meas]
        ax2.scatter(times_hrs, range_rates,
                    label=station_name, color=color, s=3,
                    alpha=0.6)

ax2.set_ylabel('Range Rate (m/s)', fontsize=11)
ax2.set_xlabel('Time (hours)', fontsize=11)
ax2.legend(loc='best')
ax2.grid(True, alpha=0.3)
ax2.set_title('Range Rate (Doppler) Measurements',
              fontsize=12, fontweight='bold')

# Plot 3: Measurement gaps/coverage
ax3 = fig.add_subplot(gs[1, :])
for i, station_name in enumerate(colors.keys()):
    station_meas = [m for m in measurements if
                    m['station'] == station_name]
    if station_meas:
        times_hrs = [m['time']/3600 for m in
                     station_meas]
        y_vals = [i] * len(times_hrs)
        ax3.scatter(times_hrs, y_vals,
                    color=colors[station_name], s=10, alpha=0.8)

ax3.set_yticks(range(len(colors)))
ax3.set_yticklabels(list(colors.keys()))
ax3.set_xlabel('Time (hours)', fontsize=11)
ax3.set_title('Measurement Availability Timeline',
              fontsize=12, fontweight='bold')
ax3.grid(True, alpha=0.3, axis='x')

# Plot 4: Trajectory with measurement locations
ax4 = fig.add_subplot(gs[2, 0])
ax4.plot(trajjectory[:, 0], trajjectory[:, 1], 'gray',
        linewidth=0.5, alpha=0.3, label='Full Trajectory')

```



```

# Plot measurement points
for station_name, color in colors.items():
    station_meas = [m for m in measurements if
                     m['station'] == station_name]
    if station_meas:
        meas_x = [m['spacecraft_pos'][0] for m in
                   station_meas]
        meas_y = [m['spacecraft_pos'][1] for m in
                   station_meas]
        ax4.scatter(meas_x, meas_y, color=color, s=5,
                     alpha=0.5, label=f'{station_name} meas')

ax4.plot(-cr3bp.mu_2, 0, 'go', markersize=12,
         label='Earth', zorder=10)
ax4.plot(cr3bp.mu_1, 0, 'ks', markersize=10,
         label='Moon', zorder=10)
ax4.set_xlabel('X (ND)', fontsize=11)
ax4.set_ylabel('Y (ND)', fontsize=11)
ax4.set_title('Measurement Locations on Trajectory',
              fontsize=12, fontweight='bold')
ax4.legend(loc='best', fontsize=8)
ax4.grid(True, alpha=0.3)
ax4.axis('equal')

# Plot 5: Measurement count histogram
ax5 = fig.add_subplot(gs[2, 1])
station_counts = [sum(1 for m in measurements if
                      m['station'] == name) for name in colors.keys()]
bars = ax5.bar(list(colors.keys()), station_counts,
               color=list(colors.values()), alpha=0.7,
               edgecolor='black')
ax5.set_ylabel('Number of Measurements', fontsize=11)
ax5.set_title('Measurements per Station', fontsize=12,
              fontweight='bold')
ax5.grid(True, alpha=0.3, axis='y')

# Add labels on bars
for bar in bars:
    height = bar.get_height()
    ax5.text(bar.get_x() + bar.get_width()/2., height,
             f'{int(height)}', ha='center', va='bottom',
             fontsize=10)

plt.savefig('measurement_coverage.png', dpi=150,
           bbox_inches='tight')
print(" Saved plot to 'measurement_coverage.png'")
plt.show()

print("\n" + "="*60)

```

```
print("MEASUREMENT SIMULATION COMPLETE")  
print("="*60)
```

C BLS.mlx — Batch Least Squares Estimator

C.1 Main Script (Setup, Loading Data, Normalization)

```
clc; close all; clear;
load('measurements.mat');
measurements = cell2mat(measurements);

% Start clock for runtime analysis
tic;

% CR3BP normalization constants for Earth-Moon system
LU = 3.844e8; % Distance unit: Earth-Moon distance in
    meters
TU = 375194.54534419806; % Time unit: characteristic time
    in seconds
VU = LU / TU; % Velocity unit: ~1025 m/s

% Convert measurements from meters to normalized units
z_range_m = [measurements.range];
z_doppler_m_s = [measurements.range_rate];

% NORMALIZE THE MEASUREMENTS
z_range = z_range_m / LU; % meters to normalized distance
z_doppler = z_doppler_m_s / VU; % m/s to normalized
    velocity

z = [z_range; z_doppler]';

% NORMALIZE THE TIMES
t = [measurements.time];
t = t / TU;

r_s_list = zeros(length(t), 3);
v_s_list = zeros(length(t), 3);
for i = 1:length(t)
    r_s_list(i, :) = measurements(i).station_pos;
    v_s_list(i, :) = measurements(i).station_vel;
end

mu = 0.012150664267188907;
mu1 = 1 - mu;
mu2 = mu;

% Initial reference state (already in normalized units)
% True initial state
x_0_ref = [-0.02684711; 0.01291372; 0.0; % pos
    3.53554434; 9.53918875; 0.0]; % vel
```

```

% Perturbed reference state
pos_pert = 1000 % m
vel_pert = 100 % m/s
x_0_ref = x_0_ref - [pos_pert/LU; pos_pert/LU; pos_pert/LU;
                    vel_pert/VU; vel_pert/VU; vel_pert/VU]

% Measurement noise covariances (convert to normalized
    units)
sigma_range_m = 1; % m
sigma_dopp_m_s = 1e-4; % m/s
sigma_range = sigma_range_m / LU; % Normalize
sigma_dopp = sigma_dopp_m_s / VU; % Normalize

Pvv = [sigma_range^2, 0; 0, sigma_dopp^2];
Pww = diag([1e-18, 1e-18, 1e-18, 1e-20, 1e-20, 1e-20]); %
    position & velocity (normalized units^2); not used in BLS

% Convergence Parameters
tol = 1e-9; % Relaxed tolerance initially
max_iter = 500;

% Storage for Diagnostics
all_residuals = zeros(length(t), max_iter);
dx_norms = zeros(max_iter, 1);
all_rms = zeros(max_iter, 1);

% CR3BP Dynamics Function
f = @(t, x) cr3bp_dynamics(t, x, mu1, mu2);
F = @(t, x) cr3bp_jacobian(t, x, mu1, mu2);

% Measurement Model Functions
h = @(x, t_k, r_s, v_s) measurement_model(x, t_k, r_s, v_s);
H_tilde = @(x, t_k, r_s, v_s) measurement_jacobian(x, t_k,
    r_s, v_s);

%% Batch Least Squares Iteration
fprintf('\n=== STARTING BATCH LEAST SQUARES ===\n');

% Removing measurements for resilience check
cut_data = true;
start_time = 0.03;
end_time = 0.43;

```

C.2 Batch Least Squares Iteration Loop

```

for n = 1:max_iter
    % Initialize accumulators
    lambda = zeros(6, 1);

```

```

Lambda = zeros(6, 6);

% Initialize propagation
Phi_prev = eye(6);
t_prev = t(1);
x_prev = x_0_ref;

rms = 0;
max_residual = 0;

% Loop through all measurements
for k = 1:length(t)
    if cut_data && t(k) > start_time && t(k) < end_time
        continue;
    end
    t_k = t(k);
    z_k = z(k, :)';

    % Get ground station position & velocity at time t_k
    r_s = r_s_list(k, :)';
    v_s = v_s_list(k, :)';

    % Propagate state and STM from previous time to
    % current time
    if k == 1
        x_k = x_0_ref;
        Phi_k = eye(6);
    else
        try
            [x_k, Phi_k] = integrate_state_stm(x_prev,
                Phi_prev, t_prev, t_k, f, F);
        catch ME
            fprintf('Integration failed at k=%d,
                t=%.2f: %s\n', k, t_k, ME.message);
            fprintf('State before failure: [%.6e, %.6e,
                %.6e, %.6e, %.6e, %.6e]\n', x_prev);
            error('Cannot continue - integration
                diverged');
        end
    end

    % Check for NaN or Inf
    if any(isnan(x_k)) || any(isinf(x_k))
        fprintf('NaN/Inf detected at k=%d\n', k);
        error('State became NaN or Inf');
    end

    % Compute predicted measurement
    z_pred = h(x_k, t_k, r_s, v_s);

```

```

% Compute residual
dz = z_k - z_pred;
all_residuals(k, n) = norm(dz);
max_residual = max(max_residual, norm(dz));

% Accumulate RMS
rms = rms + dz' * (Pvv \ dz);

% Compute measurement Jacobian
H_k = H_tilde(x_k, t_k, r_s, v_s);

% Check Jacobian
if any(isnan(H_k(:))) || any(isinf(H_k(:)))
    fprintf('Bad Jacobian at k=%d\n', k);
    error('Jacobian became NaN or Inf');
end

% Accumulate normal equations
H_Phi = H_k * Phi_k;
lambda = lambda + H_Phi' * (Pvv \ dz);
Lambda = Lambda + H_Phi' * (Pvv \ H_Phi);

% Update for next iteration
x_prev = x_k;
Phi_prev = Phi_k;
t_prev = t_k;
end

% Check condition number of Lambda
cond_Lambda = cond(Lambda);
if cond_Lambda > 1e15
    fprintf('WARNING: Lambda is poorly conditioned
            (cond = %.2e)\n', cond_Lambda);
end

% Solve for state correction
dx_0 = Lambda \ lambda;
Pxx_0 = inv(Lambda);

% Compute RMS
rms = sqrt(rms / length(t));

% Store diagnostics
dx_norms(n) = norm(dx_0);
all_rms(n) = rms;

% Display detailed progress
fprintf('Iter %2d: ||dx||=%.3e, RMS=%.3e, max_res=%.3e,
        cond(Lambda)=%.2e\n', ...
        n, norm(dx_0), rms, max_residual, cond_Lambda);

```

```

fprintf('          dx=[%.3e, %.3e, %.3e, %.3e, %.3e,
          %.3e]\n', dx_0);

% Check for divergence
if n > 1 && dx_norms(n) > 100 * dx_norms(n-1)
    fprintf('\nDIVERGING - stopping !!!\n');
    break;
end

% Check convergence
if n > 1
    if dx_norms(n) > dx_norms(n-1) && dx_norms(n-1) <
        tol
        fprintf('\nConverged after %d iterations\n', n);
        fprintf('Final RMS: %.6e\n', rms);
        fprintf('\nReached machine precision at
            iteration %d\n', n-1);
        break;
    end
end

% Update reference state with line search
% If diverging, try smaller steps
alpha = 1.0;
if n > 1 && dx_norms(n) > 0.5 * dx_norms(1)
    alpha = 0.5; % Take smaller steps if not improving
                  fast enough
    fprintf('          Using line search with
        alpha=%.2f\n', alpha);
end
if n > 1 && dx_norms(n) > dx_norms(n-1)
    alpha = 0.1; % Even smaller if getting worse
    fprintf('          Correction getting larger! Using
        alpha=%.2f\n', alpha);
end

x_0_ref_old = x_0_ref;
x_0_ref = x_0_ref + alpha * dx_0;
dx_0_old = dx_0;

% Divergence check
if norm(x_0_ref(1:3)) > 2.0 || norm(x_0_ref(4:6)) > 20.0
    fprintf('\nState became unreasonable\n');
    fprintf('Position norm: %.3f (should be < 2.0 in
        CR3BP)\n', norm(x_0_ref(1:3)));
    fprintf('Velocity norm: %.3f (should be < 20.0 in
        CR3BP)\n', norm(x_0_ref(4:6)));
    fprintf('Reverting to previous state and
        stopping.\n');
    x_0_ref = x_0_ref_old;

```

```

        break;
    end
end

```

C.3 Diagnostics and Final Estimation Output

```

% Final Results
disp('Estimate Values')
final_state = integrate_state_stm(x_0_ref, eye(6), t(1),
    t(end), f, F)
final_range_range_rate = measurement_model(final_state,
    measurements(end).time, measurements(end).station_pos',
    measurements(end).station_vel')

% Load true final state
disp('True Values')
x_true_final = [measurements(end).spacecraft_pos(:);
    measurements(end).spacecraft_vel(:)]
fprintf('\nTrue final measurements (ND):\n');
disp([measurements(end).range_true / LU;
    measurements(end).range_rate_true / VU]);

fprintf('\n=====');
fprintf('Final Estimated Initial State:\n');
fprintf('x = %.10f\n', x_0_ref(1));
fprintf('y = %.10f\n', x_0_ref(2));
fprintf('z = %.10f\n', x_0_ref(3));
fprintf('vx = %.10f\n', x_0_ref(4));
fprintf('vy = %.10f\n', x_0_ref(5));
fprintf('vz = %.10f\n', x_0_ref(6));
fprintf('\nCovariance (diagonal):\n');
fprintf('sigma = [%.3e, %.3e, %.3e, %.3e, %.3e, %.3e]\n',
    sqrt(diag(Pxx_0)));

% Compute errors
pos_error = norm(final_state(1:3) - x_true_final(1:3)) *
    LU; % meters
vel_error = norm(final_state(4:6) - x_true_final(4:6)) *
    VU; % m/s

fprintf('\n=====');
fprintf('\n=== BLS PERFORMANCE ===\n');
fprintf('Position error: %.2f m\n', pos_error);
fprintf('Velocity error: %.4f m/s\n', vel_error);

% End clock for runtime analysis
fprintf('\n=====');

```



```

fprintf('Run Time Analysis')
toc;

% Plotting
figure;

semilogy(1:n, dx_norms(1:n), 'o-', 'LineWidth', 2);
grid on;
xlabel('Iteration');
ylabel('||dx||');
title('Convergence: State Correction Norm');

semilogy(1:n, all_rms(1:n), 'o-', 'LineWidth', 2);
grid on;
xlabel('Iteration');
ylabel('RMS');
title('RMS Residual');

plot(1:length(t), all_residuals(:, n), 'LineWidth', 1.5);
grid on;
xlabel('Measurement Number');
ylabel('Residual Norm');
title('Final Residuals');

semilogy(1:min(n,10), dx_norms(1:min(n,10)), 'o-',
'LineWidth', 2);
grid on;
xlabel('Iteration');
ylabel('||dx||');
title('First 10 Iterations (linear scale)');

```

C.4 Helper Function: cr3bp_dynamics

```

function dxdt = cr3bp_dynamics(~, x, mu1, mu2)
    r1 = sqrt((x(1) + mu2)^2 + x(2)^2 + x(3)^2);
    r2 = sqrt((x(1) - mu1)^2 + x(2)^2 + x(3)^2);

    dxdt = zeros(6,1);
    dxdt(1) = x(4);
    dxdt(2) = x(5);
    dxdt(3) = x(6);
    dxdt(4) = 2*x(5) + x(1) - mu1*(x(1)+mu2)/r1^3 -
        mu2*(x(1)-mu1)/r2^3;
    dxdt(5) = -2*x(4) + x(2) - mu1*x(2)/r1^3 -
        mu2*x(2)/r2^3;
    dxdt(6) = -mu1*x(3)/r1^3 - mu2*x(3)/r2^3;
end

```

C.5 Helper Function: cr3bp_jacobian

```
function A = cr3bp_jacobian(~, x, mu1, mu2)
    r1 = sqrt((x(1) + mu2)^2 + x(2)^2 + x(3)^2);
    r2 = sqrt((x(1) - mu1)^2 + x(2)^2 + x(3)^2);

    Uxx = 1 - mu1/r1^3 - mu2/r2^3 + 3*mu1*(x(1)+mu2)^2/r1^5
          + 3*mu2*(x(1)-mu1)^2/r2^5;
    Uyy = 1 - mu1/r1^3 - mu2/r2^3 + 3*mu1*x(2)^2/r1^5 +
          3*mu2*x(2)^2/r2^5;
    Uzz = -mu1/r1^3 - mu2/r2^3 + 3*mu1*x(3)^2/r1^5 +
          3*mu2*x(3)^2/r2^5;
    Uxy = 3*mu1*(x(1)+mu2)*x(2)/r1^5 +
          3*mu2*(x(1)-mu1)*x(2)/r2^5;
    Uxz = 3*mu1*(x(1)+mu2)*x(3)/r1^5 +
          3*mu2*(x(1)-mu1)*x(3)/r2^5;
    Uyz = 3*mu1*x(2)*x(3)/r1^5 + 3*mu2*x(2)*x(3)/r2^5;

    A = [0, 0, 0, 1, 0, 0;
         0, 0, 0, 0, 1, 0;
         0, 0, 0, 0, 0, 1;
         Uxx, Uxy, Uxz, 0, 2, 0;
         Uxy, Uyy, Uyz, -2, 0, 0;
         Uxz, Uyz, Uzz, 0, 0, 0];
end
```

C.6 Helper Function: measurement_model

```
function z = measurement_model(x, t_k, r_s, v_s)
    pos = x(1:3);
    vel = x(4:6);

    dr = pos - r_s;
    dv = vel - v_s;

    range = norm(dr);
    range_rate = dot(dr, dv) / range;

    z = [range; range_rate];
end
```

C.7 Helper Function: measurement_jacobian

```
function H = measurement_jacobian(x, t_k, r_s, v_s)
    pos = x(1:3);
    vel = x(4:6);
```

```

dr = pos - r_s;
dv = vel - v_s;

range = norm(dr);

% Range Jacobian
H_range = [dr'/range, zeros(1,3)];

% Range rate Jacobian
H_rr_pos = (dv'*range - dr'*dot(dr,dv)/range) / range^2;
H_rr_vel = dr' / range;
H_range_rate = [H_rr_pos, H_rr_vel];

H = [H_range; H_range_rate];
end

```

C.8 Helper Function: integrate_state_stm

```

function [x_final, Phi_final] = integrate_state_stm(x0,
Phi0, t0, tf, f, F)
if abs(tf - t0) < 1e-12
    x_final = x0;
    Phi_final = Phi0;
    return;
end

y0 = [x0; reshape(Phi0, 36, 1)];

% Use more relaxed tolerances to avoid stiffness issues
options = odeset('RelTol', 1e-10, 'AbsTol', 1e-12,
'MaxStep', 0.1);

try
    [~, y] = ode113(@(t,y) combined_dynamics(t, y, f,
F), [t0, tf], y0, options);
catch
    warning('ode113 failed, switching to ode15s');
    [~, y] = ode15s(@(t,y) combined_dynamics(t, y, f,
F), [t0, tf], y0, options);
end

y_final = y(end, :);
x_final = y_final(1:6);
Phi_final = reshape(y_final(7:42), 6, 6);
end

```

C.9 Helper Function: combined_dynamics

```
function dydt = combined_dynamics(t, y, f, F)
    x = y(1:6);
    Phi = reshape(y(7:42), 6, 6);

    dx = f(t, x);
    A = F(t, x);
    dPhi = A * Phi;

    dydt = [dx; reshape(dPhi, 36, 1)];
end
```

D UKF.mlx — Unscented Kalman Filter

D.1 Main Script: Initialization, Normalization, and Setup

```
clc; close all; clear;
load('measurements.mat');
measurements = cell2mat(measurements);

% Start clock for runtime analysis
tic;

% CR3BP normalization constants for Earth-Moon system
LU = 3.844e8; % Distance unit: Earth-Moon distance in
    meters
TU = 375194.54534419806; % Time unit: characteristic time
    in seconds
VU = LU / TU; % Velocity unit: ~1025 m/s

% Convert measurements from meters to normalized units
z_range_m = [measurements.range];
z_doppler_m_s = [measurements.range_rate];

% NORMALIZE THE MEASUREMENTS
z_range = z_range_m / LU; % meters to normalized distance
z_doppler = z_doppler_m_s / VU; % m/s to normalized
    velocity

z = [z_range; z_doppler]';

% NORMALIZE THE TIMES
t = [measurements.time];
t = t / TU;

r_s_list = zeros(length(t), 3);
v_s_list = zeros(length(t), 3);
for i = 1:length(t)
    r_s_list(i, :) = measurements(i).station_pos;
    v_s_list(i, :) = measurements(i).station_vel;
end

mu = 0.012150664267188907;
mu1 = 1 - mu;
mu2 = mu;

% Initial reference state (already in normalized units)
% True initial state
x_0_ref = [-0.02684711; 0.01291372; 0.0; % pos
    3.53554434; 9.53918875; 0.0]; % vel
```

```

% Perturbed reference state
pos_pert = 1000; % m
vel_pert = 100; % m/s
x_0_ref = x_0_ref - [pos_pert/LU; pos_pert/LU; pos_pert/LU;
                    vel_pert/VU; vel_pert/VU; vel_pert/VU]
sigma_pos_pert = pos_pert / LU;
sigma_vel_pert = vel_pert / VU;

Pxx0 = diag([sigma_pos_pert^2, sigma_pos_pert^2,
            sigma_pos_pert^2, ...
            sigma_vel_pert^2, sigma_vel_pert^2,
            sigma_vel_pert^2]);

% Measurement noise covariances (convert to normalized
    units)
sigma_range_m = 1; % m
sigma_dopp_m_s = 1e-4; % m/s
sigma_range = sigma_range_m / LU; % Normalize
sigma_dopp = sigma_dopp_m_s / VU; % Normalize

Pvv = [sigma_range^2, 0; 0, sigma_dopp^2];
Q = diag([5e-20 * ones(1,3), 5e-22 * ones(1,3)]); %
    position & velocity (normalized units^2)

% CR3BP Dynamics Function
f = @(t, x) cr3bp_dynamics(t, x, mu1, mu2);
F = @(t, x) cr3bp_jacobian(t, x, mu1, mu2);

% Measurement Model Functions
h = @(x, t_k, r_s, v_s) measurement_model(x, t_k, r_s, v_s);

% UKF Setup
n = 6;
beta = 2;
kappa_param = 0;
alpha = 1;
lambda = alpha^2 * (n + kappa_param) - n;

w_m = zeros(1, 2*n + 1);
w_c = zeros(1, 2*n + 1);

w_m(1) = lambda / (n + lambda);
w_m(2:end) = 1 / (2 * (n + lambda));

w_c(1) = lambda / (n + lambda) + (1 - alpha^2 + beta);
w_c(2:end) = 1 / (2 * (n + lambda));

% Initialize storage
N_steps = length(t);
mx_filtered = zeros(n, N_steps);

```

```

P_filtered = zeros(n, N_steps);
mx_predicted = zeros(n, N_steps);
P_predicted = zeros(n, N_steps);
estimation_error = zeros(n, N_steps);

% Start UKF
mxkm1 = x_0_ref;
Pxxkm1 = Pxx0;
mx_predicted(:, 1) = mxkm1;
mx_filtered(:, 1) = mxkm1;
P_predicted(:, 1) = diag(Pxxkm1);
P_filtered(:, 1) = diag(Pxxkm1);
x_true_initial = [measurements(1).spacecraft_pos(:);
    measurements(1).spacecraft_vel(:)];
estimation_error(:, 1) = x_true_initial - mxkm1;

% Removing measurements for resilience check
cut_data = true;
start_time = 0.003;
end_time = 0.43;

```

D.2 UKF Time Update and Measurement Update Loop

```

fprintf('\n=== STARTING UNSCENTED KALMAN FILTER ===\n');
for k = 2:length(t)
    t_k = t(k);
    tkm1 = t(k-1);
    z_k = z(k, :)';

    % Get ground station position & velocity at time t_k
    r_s = r_s_list(k, :)';
    v_s = v_s_list(k, :)';

    chi = zeros(6, 2*n + 1);
    % Generate sigma points
    chi(:, 1) = mxkm1; % Mean state
    sqrtP = chol(Pxxkm1, 'lower'); % Cholesky decomposition
    for i = 1:n
        chi(:, i + 1) = mxkm1 + sqrt(n + lambda) * sqrtP(:, i);
        chi(:, n + i + 1) = mxkm1 - sqrt(n + lambda) * sqrtP(:, i);
    end

    % Propagate sigma points through dynamics
    for i = 1:(2*n + 1)
        chi(:, i) = integrate_state(chi(:, i), tkm1, t_k, f);
    end
end

```

```

end

% Compute Predicted mean and cov
mxkm = sum(w_m .* chi, 2);
Pxxkm = 0;
for i = 1:(2*n + 1)
    Pxxkm = Pxxkm + w_c(i) * (chi(:, i) - mxkm) *
        (chi(:, i) - mxkm)';
end
% Very small process noise - accounts for numerical
    integration errors

Pxxkm = Pxxkm + Q;

% Store predicted values
mx_predicted(:, k) = mxkm;
P_predicted(:, k) = diag(Pxxkm);

if cut_data && t(k) > start_time && t(k) < end_time
    mxkp = mxkm;
    Pxxkp = Pxxkm;
else
    % Propagate sigma points through measurement model
    Z = zeros(2, 2*n + 1);
    for i = 1:(2*n + 1)
        Z(:, i) = h(chi(:, i), t_k, r_s, v_s);
    end

    % Calculate Filtered Values
    mzkm = sum(w_m .* Z, 2);
    Pzzkm = 0;
    Pxzkm = 0;
    for i = 1:(2*n + 1)
        Pzzkm = Pzzkm + w_c(i) * (Z(:, i) - mzkm) *
            (Z(:, i) - mzkm)';
        Pxzkm = Pxzkm + w_c(i) * (chi(:, i) - mxkm) *
            (Z(:, i) - mzkm)';
    end
    Pzzkm = Pzzkm + Pvv;

    Kk = Pxzkm / Pzzkm;
    mxkp = mxkm + Kk * (z_k - mzkm);
    Pxxkp = Pxxkm - Kk * Pzzkm * Kk';
end

% Store filtered values
mx_filtered(:, k) = mxkp;
P_filtered(:, k) = diag(Pxxkp);

% Compute error

```



```

        x_true_k = [measurements(k).spacecraft_pos(:);
                    measurements(k).spacecraft_vel(:)];
        estimation_error(:, k) = x_true_k - mxkp;

        % Propagate for next iteration
        mxkm1 = mxkp;
        Pxxkm1 = Pxxkp;

end

```

D.3 Final State Evaluation and Error Metrics

```

% Final state check
fprintf('\n=== UKF FINAL ESTIMATE ===\n');
disp(mxkp);
fprintf('Predicted final measurement (ND):\n');
disp(measurement_model(mxkp, measurements(end).time / TU,
    measurements(end).station_pos',
    measurements(end).station_vel'));
fprintf('Final covariance (P):\n');
disp(Pxxkp);

fprintf('\nTrue final (ND):\n');
x_true_final = [measurements(end).spacecraft_pos(:);
    measurements(end).spacecraft_vel(:)];
disp(x_true_final);

fprintf('\nTrue final measurements (ND):\n');
disp([measurements(end).range_true / LU;
    measurements(end).range_rate_true / VU]);

% Compute errors
pos_error = norm(mxkp(1:3) - x_true_final(1:3)) * LU; %
    meters
vel_error = norm(mxkp(4:6) - x_true_final(4:6)) * VU; % m/s

fprintf('\n=== UKF PERFORMANCE ===\n');
fprintf('Data Missing Mode: %s', string(cut_data));
fprintf('Position error: %.2f m\n', pos_error);
fprintf('Velocity error: %.4f m/s\n', vel_error);

% 3-sigma interval
pos_3sigma = 3 * sqrt(trace(Pxxkp(1:3,1:3))) * LU;
vel_3sigma = 3 * sqrt(trace(Pxxkp(4:6,4:6))) * VU;

fprintf('Position 3sigma interval: %.2f m\n', pos_3sigma);
fprintf('Velocity 3sigma interval: %.4f m/s\n', vel_3sigma);

```

```

fprintf('Position error within 3sigma? %s\n',
        string(pos_error < pos_3sigma));
fprintf('Velocity error within 3sigma? %s\n',
        string(vel_error < vel_3sigma));

% End clock for runtime analysis
fprintf('\n=====');
fprintf('Run Time Analysis')
toc;

```

D.4 Plotting Section

```

% Plotting

figure;
hold on;
plot(t, estimation_error(1, :));
plot(t, 3 * sqrt(P_filtered(1, :)), 'r-');
plot(t, -3 * sqrt(P_filtered(1, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('X Position Error', '\pm 3\sigma');
title('UKF r_x Estimation Error with \pm 3\sigma');

figure;
hold on;
plot(t, estimation_error(2, :));
plot(t, 3 * sqrt(P_filtered(2, :)), 'r-');
plot(t, -3 * sqrt(P_filtered(2, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('Y Position Error', '\pm 3\sigma');
title('UKF r_y Estimation Error with \pm 3\sigma');

figure;
hold on;
plot(t, estimation_error(3, :));
plot(t, 3 * sqrt(P_filtered(3, :)), 'r-');
plot(t, -3 * sqrt(P_filtered(3, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('Z Position Error', '\pm 3\sigma');
title('UKF r_z Estimation Error with \pm 3\sigma');

figure;
hold on;
plot(t, estimation_error(4, :));
plot(t, 3 * sqrt(P_filtered(4, :)), 'r-');

```

```

plot(t, -3 * sqrt(P_filtered(4, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('X Velocity Error', '\pm 3\sigma');
title('UKF v_x Estimation Error with \pm 3\sigma');

figure;
hold on;
plot(t, estimation_error(5, :));
plot(t, 3 * sqrt(P_filtered(5, :)), 'r-');
plot(t, -3 * sqrt(P_filtered(5, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('Y Velocity Error', '\pm 3\sigma');
title('UKF v_y Estimation Error with \pm 3\sigma');

figure;
hold on;
plot(t, estimation_error(6, :));
plot(t, 3 * sqrt(P_filtered(6, :)), 'r-');
plot(t, -3 * sqrt(P_filtered(6, :)), 'r-');
xlabel('Time (ND)');
ylabel('Error (ND)');
legend('Z Velocity Error', '\pm 3\sigma');
title('UKF v_z Estimation Error with \pm 3\sigma');

```

D.5 Helper Function: cr3bp_dynamics

```

function dxdt = cr3bp_dynamics(~, x, mu1, mu2)
    r1 = sqrt((x(1) + mu2)^2 + x(2)^2 + x(3)^2);
    r2 = sqrt((x(1) - mu1)^2 + x(2)^2 + x(3)^2);
    dxdt = zeros(6,1);
    dxdt(1) = x(4);
    dxdt(2) = x(5);
    dxdt(3) = x(6);
    dxdt(4) = 2*x(5) + x(1) - mu1*(x(1)+mu2)/r1^3 -
        mu2*(x(1)-mu1)/r2^3;
    dxdt(5) = -2*x(4) + x(2) - mu1*x(2)/r1^3 -
        mu2*x(2)/r2^3;
    dxdt(6) = -mu1*x(3)/r1^3 - mu2*x(3)/r2^3;
end

```

D.6 Helper Function: cr3bp_jacobian

```

function A = cr3bp_jacobian(~, x, mu1, mu2)
    r1 = sqrt((x(1) + mu2)^2 + x(2)^2 + x(3)^2);
    r2 = sqrt((x(1) - mu1)^2 + x(2)^2 + x(3)^2);

```

```

Uxx = 1 - mu1/r1^3 - mu2/r2^3 + 3*mu1*(x(1)+mu2)^2/r1^5
      + 3*mu2*(x(1)-mu1)^2/r2^5;
Uyy = 1 - mu1/r1^3 - mu2/r2^3 + 3*mu1*x(2)^2/r1^5 +
      3*mu2*x(2)^2/r2^5;
Uzz = -mu1/r1^3 - mu2/r2^3 + 3*mu1*x(3)^2/r1^5 +
      3*mu2*x(3)^2/r2^5;
Uxy = 3*mu1*(x(1)+mu2)*x(2)/r1^5 +
      3*mu2*(x(1)-mu1)*x(2)/r2^5;
Uxz = 3*mu1*(x(1)+mu2)*x(3)/r1^5 +
      3*mu2*(x(1)-mu1)*x(3)/r2^5;
Uyz = 3*mu1*x(2)*x(3)/r1^5 + 3*mu2*x(2)*x(3)/r2^5;

A = [0, 0, 0, 1, 0, 0;
      0, 0, 0, 0, 1, 0;
      0, 0, 0, 0, 0, 1;
      Uxx, Uxy, Uxz, 0, 2, 0;
      Uxy, Uyy, Uyz, -2, 0, 0;
      Uxz, Uyz, Uzz, 0, 0, 0];
end

```

D.7 Helper Function: measurement_model

```

function z = measurement_model(x, t_k, r_s, v_s)
    pos = x(1:3);
    vel = x(4:6);

    dr = pos - r_s;
    dv = vel - v_s;

    range = norm(dr);
    range_rate = dot(dr, dv) / range;

    z = [range; range_rate];
end

```

D.8 Helper Function: integrate_state

```

function x_final = integrate_state(x0, t0, tf, f)
    if abs(tf - t0) < 1e-12
        x_final = x0;
        return;
    end

    y0 = x0;

    % Use more relaxed tolerances to avoid stiffness issues

```

```

options = odeset('RelTol', 1e-10, 'AbsTol', 1e-12,
    'MaxStep', 0.1);

try
    [~, y] = ode113(@(t,y) combined_dynamics(t, y, f),
        [t0, tf], y0, options);
catch
    warning('ode113 failed, switching to ode15s');
    [~, y] = ode15s(@(t,y) combined_dynamics(t, y, f),
        [t0, tf], y0, options);
end

y_final = y(end, :);
x_final = y_final(1:6);
end

```

D.9 Helper Function: combined_dynamics

```

function dydt = combined_dynamics(t, y, f)
    x = y(1:6);

    dx = f(t, x);

    dydt = dx;
end

```

References

- [1] Segan, S. and Marceta, D., “Orbit Determination and Parameter Estimation: Extended Kalman Filter (EKF) Versus Least Squares Orbit Determination (LSQOD),” *Publications de l’Observatoire Astronomique de Beograd*, 2009.
- [2] Oza, D. H. and Mistretta, G. D., “Comparison of ERBS Orbit Determination Accuracy Using Batch Least-Squares and Sequential Methods,” Tech. rep., NASA, 1991.
- [3] Wu, J. and Huang, C., “Unscented Kalman filter and its application in GPS-based satellite orbit determination,” *Acta Astronomica Sinica*, 2005, pp. 55–61.
- [4] Koon, W. S., Lo, M. W., Marsden, J. E., and Ross, S. D., *Dynamical Systems, the Three-Body Problem and Space Mission Design*, Marsden Books, 2022.
- [5] Berner, J. B., Bryant, S. H., and Kinman, P. W., “Range Measurement as Practiced in the Deep Space Network,” *Proceedings of the IEEE*, Vol. 95, No. 11, 2007, pp. 2202–2214.
- [6] Buccino, D. R., Oudrhiri, K., Parisi, M., Park, R. S., Mazarico, E., Tortora, P., Withers, P., Genova, A., and Zannoni, M., “Precision of Spacecraft Doppler Tracking at Low Signal-to-Noise Ratios,” *Radio Science*, Vol. 58, No. 11, 2023.
- [7] Burke, J. V., “Backtracking Line Search,” Lecture notes, Department of Mathematics, University of Washington, 2016.
- [8] Crassidis, J. L. and Markley, F. L., “Unscented Filtering for Spacecraft Attitude Estimation,” *Journal of Guidance, Control, and Dynamics*, Vol. 26, No. 4, 2003, pp. 536–542.
- [9] Crassidis, J., “Sigma-Point Kalman Filtering for Integrated GPS and Inertial Navigation,” *Aerospace and Electronic Systems, IEEE Transactions on*, Vol. 42, 05 2006, pp. 750 – 756.
- [10] Monaghan, H., “Where is the DSN Located?” NASA website, Mar 30 2020.
- [11] a.i. solutions, “Station Geodetics File (groundstations.dat),” n.d.